

V_Carpool — Project Mastery for Technical Interviews

Satya1929/Carpool1_App | June 2026 | Python · Streamlit · Google Sheets · GitHub Actions

- Legend:
- **FACT** — Directly observed in source code / configuration
 - **INFERENCE** — Reasonable conclusion from evidence
 - **ASSUMPTION** — Not confirmed; stated explicitly
 - **RECOMMENDATION** — Suggested improvement

Table of Contents

1. Project Motivation
2. Functional Requirements
3. Non-Functional Requirements
4. Tech Stack Breakdown
5. System Architecture
6. Engineering Decision Analysis
7. End-to-End Data Flow
8. Database Engineering Review
9. Security Audit
10. Failure Mode Analysis
11. Performance Engineering
12. Observability Review
13. Testing Strategy Review
14. Deployment & DevOps
15. Code Quality Review
16. Production Readiness Scorecard
17. Staff Engineer Review
18. Project Learnings
19. Resume Claim Validation
20. Interview Story Generator

- 21. [Mock Interview Drill](#)
- 22. [Final Project Mastery Cheatsheet](#)
- 23. [Feature Deep Dive Cross-Examination](#)
- 24. [Code Walkthrough Preparation](#)
- 25. [Question Tree Expansion](#)
- 26. [Project Structure & Code Organization](#)
- 27. [API Deep Dive](#)
- 28. [Database Query Cross-Examination](#)
 - [Hidden Interview Traps](#)
 - [Executive Summary](#)

1. Project Motivation

High-Level Problem Statement

University students leaving campus for holidays face a coordination problem: hundreds of students travel to overlapping destinations on the same dates, but have no shared visibility into who else is traveling when and where. They either overpay for solo travel, spend hours in WhatsApp groups manually asking “anyone going to Pune on Oct 12?”, or miss carpool opportunities entirely.

Business Goal

Build a zero-friction matchmaking layer on top of existing data collection infrastructure (Google Forms + Google Sheets) to let students self-organize carpools without requiring any app installation, account creation, or complex UI.

Target Users

User Type	Pain Point
University students	No visibility into who else is traveling on the same day
Students leaving for Diwali/semester break	High travel demand in short windows; seats go fast
Cost-conscious travelers	Cab/train costs are high; split fares save 60–70%

Existing Alternatives & Why They're Insufficient

Alternative	Why Insufficient
WhatsApp group messages	Ephemeral, noisy, no structured filtering by date
BlaBlaCar / OLA Outstation	Not campus-scoped; external strangers; cost overhead
Manual spreadsheet sharing	No search; no charts; no discovery
Nothing (default)	Status quo — students just miss opportunities

Engineering Significance

- **FACT:** 1,100+ real users, 700+ form entries — this is a live production system, not a demo
- **FACT:** App served 300+ daily requests at peak (Diwali window) — per README
- Real-world usage creates real engineering challenges: API rate limits, data freshness, concurrent reads

Elevator Pitches

30-Second Pitch

"V_Carpool is a real-time travel matchmaking app used by 1,100+ university students. Students submit their travel details via Google Forms. My Streamlit app reads that data live and lets users search for travel partners by date, see time and destination distributions in charts, and reach out directly. It's zero-friction — no account needed, no app to install. At peak (Diwali break), it handled 300+ daily requests."

90-Second Interview Explanation

"The problem I solved was coordination failure during university holiday travel. Every semester break, hundreds of students travel to the same cities on the same dates — but they had no way to discover each other without spamming WhatsApp groups."

I built V_Carpool, a Python + Streamlit web app backed by Google Sheets as a live database. Students submit preferences through a Google Form — date, time, destination — and my app reads that in real-time, letting them search by date and see who else is traveling. It visualizes the data with pie charts showing popular travel times and destinations.

What makes this real: it has 1,100+ users, 5 GitHub Actions CI/CD pipelines (testing, linting, security scanning, smoke testing, and auto-documentation), and it's deployed live on Streamlit Cloud. I also wrote

unit tests for the core data logic, set up dependency vulnerability scanning with pip-audit, and secret scanning with TruffleHog.

Going forward I'm planning to add AI-powered matchmaking — scoring partners by destination similarity and time proximity — which is already on the roadmap."

Non-Technical Explanation

"Imagine you and 500 other students all want to share a cab home for Diwali. But nobody knows who's going where on which day. I built a simple website where students fill out a form with their travel plans. The site then shows everyone who else is going to the same place at the same time, so they can contact each other and share the ride. Over 1,100 students used it."

2. Functional Requirements

Core Features

Feature	Implementation	File
Home page with navigation	<code>st.header</code> , <code>st.button</code> , <code>st.link_button</code>	<code>main.py</code>
Search travelers by date	Date picker → GSheets filter → card display	<code>pages/1_🔍_Search_by_Date.py</code>
Pie chart: travel time distribution	Matplotlib pie chart of hourly bins	<code>pages/1_🔍_Search_by_Date.py</code>
Pie chart: destination distribution	Matplotlib pie chart of destinations	<code>pages/1_🔍_Search_by_Date.py</code>
All-days summary dashboard	Pie charts of all dates + all destinations	<code>pages/2_🇮🇳_All_Days_Summary.py</code>
Credits page	Developer profile card with links	<code>pages/3_🎨_Credits_Page.py</code>
Form link	External Google Form link	<code>main.py</code>
Spreadsheet link	External Google Sheet link	<code>main.py</code>

Secondary Features

- **FACT:** `st.balloons()` celebratory animation on search page
- **FACT:** Hover-animated cards via custom CSS
- **FACT:** YouTube tutorial link embedded in homepage

User Roles

Role	Permissions	Auth Required
Visitor	View all data, search by date, see all charts	No
Form submitter	Submit data via Google Form	Google account (form-side only)
Admin	None — no admin role exists	N/A

FACT: There is zero authentication in the Streamlit app itself. Any visitor can see all traveler data including names and phone numbers.

APIs Exposed

FACT: None. This is a Streamlit app — it exposes no REST endpoints, no GraphQL, no WebSocket API. All logic is server-side rendered Python.

What the System Intentionally Does NOT Do

- No real-time messaging between travelers
- No booking or payment processing
- No user account creation
- No data editing (edit happens via Google Form's "edit response" email link)
- No admin moderation panel

3. Non-Functional Requirements

NFR	Current State	Score	Notes
Scalability	Poor	2/10	Full table GSheets read per query; no horizontal scale
Reliability	Medium	5/10	Streamlit Cloud uptime ~99%; GSheets reliability ~99.9%
Availability	Medium	5/10	Single deployment; no failover
Performance	Poor	2/10	Cache actively cleared on every load; no pagination
Maintainability	Poor	3/10	Dead code, magic numbers, hardcoded URLs, no config
Security	Poor	2/10	PII exposed publicly; no auth; unsafe HTML rendering
Observability	None	1/10	Zero logging, zero metrics, zero error tracking
Cost efficiency	Excellent	9/10	Free tier: Streamlit Cloud + Google Sheets + GitHub Actions

4. Tech Stack Breakdown

Frontend: Streamlit

Dimension	Detail
What it is	Python library that renders web UIs from pure Python code. No HTML/JS/CSS required.
Why chosen	FACT: Fastest path from data to web UI for Python-first developers. No frontend framework knowledge needed.
Advantages	Zero JS, rapid prototyping, built-in widgets, easy deployment, Python-native
Disadvantages	Limited UI customization, no real SPA behavior, poor mobile UX, re-renders entire page on interaction
Alternatives	Flask + Jinja2, Django, FastAPI + React, Dash (Plotly), Panel
Tradeoffs	Speed of development vs. flexibility and interview credibility
When it fails	High-concurrency use cases; complex UI state management; custom interactive components
When another choice is better	Any production app with >1,000 concurrent users, complex auth, or real-time features

Interview Trap: "Why didn't you use React or Vue?"

Strong answer: "Streamlit was the right tool for this stage — I needed to ship a working product quickly. The users needed functionality, not a polished SPA. However, I recognize its limitations: it re-renders on every interaction, doesn't support complex auth flows, and can't scale horizontally the way a proper React + FastAPI stack can. My roadmap includes migrating the backend to FastAPI with a proper frontend."

Backend: Streamlit Server (Python)

Dimension	Detail
What it is	Streamlit runs a Tornado-based Python web server internally
Why chosen	Implicit — comes with Streamlit
Advantages	Zero configuration; handles HTTP, WebSocket, static assets
Disadvantages	No REST API exposure; tightly coupled to UI; single-threaded per session
Alternatives	FastAPI, Flask, Django REST Framework
Production gap	Cannot be consumed by mobile apps, bots, or other services

Database: Google Sheets

Dimension	Detail
What it is	Cloud spreadsheet used as a live data store
Why chosen	FACT: Google Forms writes directly to Sheets — zero backend code needed for data ingestion
Advantages	Free, zero-ops, instant form-to-data pipeline, human-readable
Disadvantages	100 req/100s API rate limit; no indexing; no transactions; full table reads only; PII exposure risk
Alternatives	SQLite, PostgreSQL, Firebase Firestore, Supabase
Tradeoffs	Operational simplicity vs. query efficiency, security, and scalability
When it fails	>50 concurrent users hammering the API; dataset >50,000 rows; need for filtered server-side queries

Interview Trap: “Isn’t Google Sheets a terrible database?”

Strong answer: “For this use case, it’s a pragmatic trade-off, not a mistake. The data ingestion pipeline — Google Forms → Sheets — required zero backend code. The dataset is small (700 entries), the read pattern is infrequent, and the operational cost is zero. I’m aware of the limitations: no indexing means every query is a full table scan, the API rate limit is 100 requests per 100 seconds, and the Sheet being public is a PII risk I’ve identified and plan to fix by restricting access to a service account only. At 10x scale, I’d migrate to PostgreSQL with a FastAPI backend.”

Data Processing: Pandas

Dimension	Detail
What it is	Python DataFrame library for data manipulation
Why chosen	FACT: Standard choice for tabular data processing in Python
Usage	Date parsing, NaN handling, filtering, value_counts, datetime formatting
Alternatives	Polars (faster), plain Python dicts, SQL queries
Tradeoffs	Familiar and well-documented vs. Polars being 5–10x faster for large datasets

Visualization: Matplotlib

Dimension	Detail
What it is	Static chart library
Why chosen	INFERENCE: Default choice; widely known; no additional dependency learning required
Disadvantages	No interactivity, no hover tooltips, no zoom, looks dated
Better alternative	Plotly Express — same effort, interactive charts, better UX
Interview note	Upgrading to Plotly is a 1-hour change with high demo value

Deployment: Streamlit Community Cloud

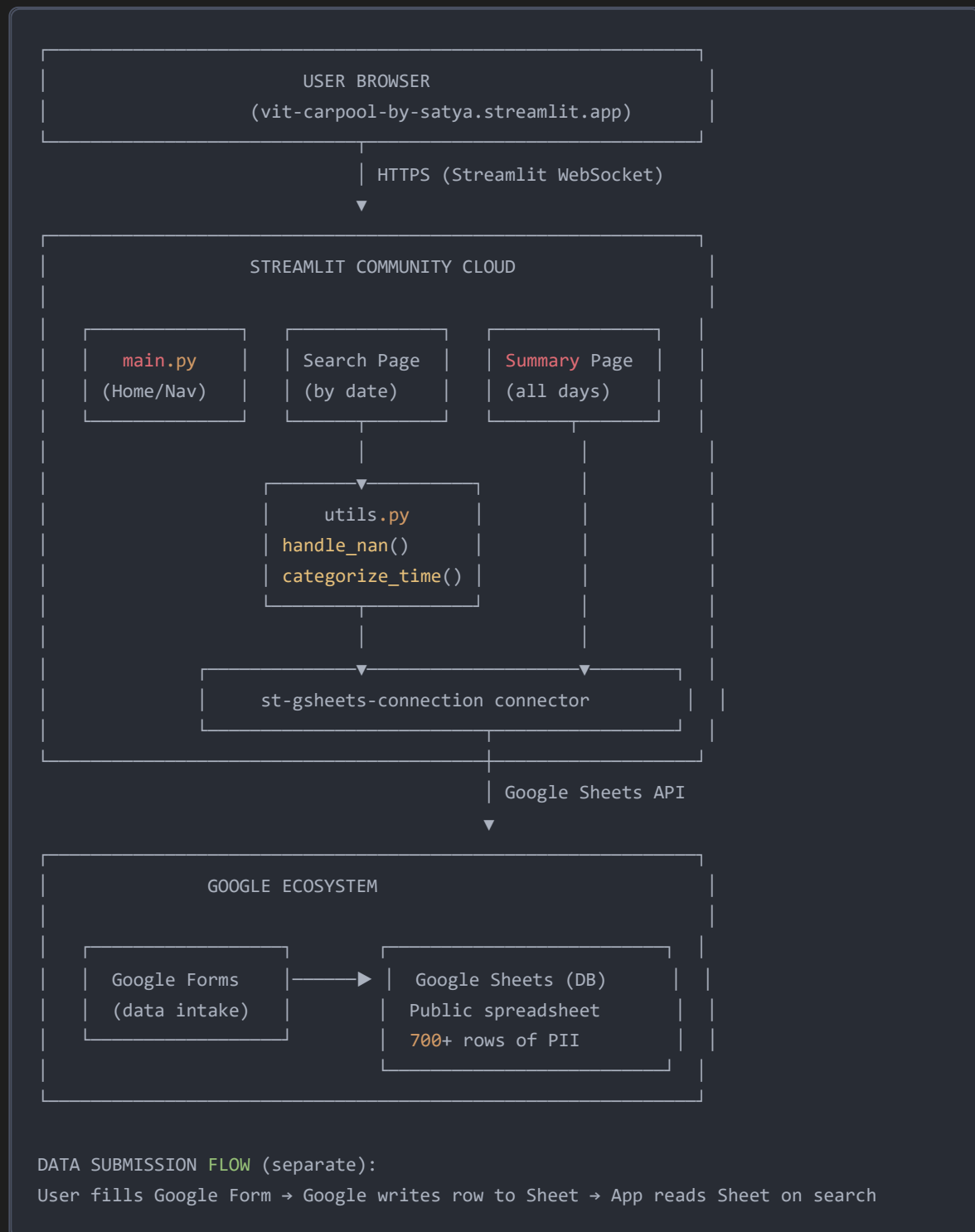
Dimension	Detail
What it is	Free hosting platform for Streamlit apps, connected to GitHub
Why chosen	Push-to-deploy, free tier, zero infrastructure management
Advantages	Zero cost, auto-deploys on git push, HTTPS, managed secrets
Disadvantages	No custom Docker, no scaling control, sleep on inactivity, no SLA
Alternatives	Railway, Render, Fly.io , Heroku, AWS EC2, GCP Cloud Run
Interview note	INFERENCE: The app may go to sleep after inactivity — cold start times can be 30–60 seconds

CI/CD: GitHub Actions

Dimension	Detail
What it is	YAML-based automation workflows on GitHub
Why chosen	FACT: Free with public repos; integrates directly with GitHub
Workflows	ci.yml, lint.yml, security-audit.yml, smoke-test.yml, update-docs.yml
Strengths	5 real pipelines; coverage artifact upload; TruffleHog secret scan
Weaknesses	FACT: <code>continue-on-error: true</code> on lint and security — failures don't block merges

5. System Architecture

A. High-Level Architecture



B. Component Diagram

```

Carpool_App/
├─ main.py           ← Entry point; home page; navigation
├─ utils.py          ← Pure utility functions (no Streamlit imports)
├─ test_app.py       ← Unit tests for utils.py
├─ requirements.txt  ← Python dependencies
├─ packages.txt       ← OS packages (empty)
├─ .devcontainer/
│   └─ devcontainer.json ← GitHub Codespaces config
├─ .github/workflows/
│   ├── ci.yml        ← pytest + coverage
│   ├── lint.yml       ← Ruff linter
│   ├── security-audit.yml ← pip-audit + TruffleHog
│   ├── smoke-test.yml ← HTTP health check post-deploy
│   └─ update-docs.yml ← Weekly auto-docs PR
└─ pages/
    ├── 1_🔍_Search_by_Date.py ← Core feature: date-based search
    ├── 2_📊_All_Days_Summary.py ← Analytics overview
    └─ 3_👤_Credits_Page.py    ← Developer info
  
```

C. Request Lifecycle

```

User selects date → clicks "Search"
    |
    ▼
Streamlit re-runs 1_🔗_Search_by_Date.py
    |
    ▼
st.cache_data.clear() ← clears ALL cached data (bug: defeats caching)
    |
    ▼
fetch_data() called
    |
    ▼
GSheetsConnection.read(spreadsheet=url, usecols=[2,3,4,5,6,7])
    |
    ▼
Google Sheets API returns full table (~700 rows)
    |
    ▼
Pandas: convert 'Travel Date' column to datetime
    |
    ▼
Pandas: filter rows where Travel Date == selected date
    |
    ▼
If results found:
    ├── Render HTML cards (one per traveler)
    ├── categorize_time() on travel times → pie chart
    └── value_counts() on destinations → pie chart
Else:
    └── st.write("Sorry, No data found")
  
```

D. Deployment Architecture

```

GitHub Repository (main branch)
  |
  | push event
  ▼
GitHub Actions (4 parallel workflows)
├─ CI: pytest --cov → uploads htmlcov/
├─ Lint: ruff check → ruff format --check
├─ Security: pip-audit + TruffleHog
├─ (on push to main) → Streamlit Cloud auto-pulls & redeploys
    |
    | 10 min later (sleep 600s)
    ▼
    smoke-test.yml:
    curl https://vit-carpool-by-satya.streamlit.app/
    → check HTTP 200
  
```

Architectural Strengths

- Serverless data ingestion (Google Forms handles all writes)
- Zero infrastructure management
- Separation of utilities into `utils.py` (testable in isolation)
- Multi-page routing via Streamlit's native `pages/` directory

Architectural Weaknesses

- **FACT:** No API layer — app cannot be consumed by mobile or third-party
- **FACT:** Database is a public spreadsheet — security anti-pattern
- **FACT:** Full table read on every query — no server-side filtering
- **FACT:** Single deployment, no failover, no load balancing
- **INFERENCE:** Streamlit session model means no shared state between users

6. Engineering Decision Analysis

Decision 1: Google Sheets as Database

Aspect	Detail
Decision	Use Google Sheets as the primary data store
Why	Forms → Sheets pipeline requires zero backend code for data ingestion
Alternative	SQLite, PostgreSQL, Firebase
Tradeoff	Zero ops vs. no indexing, rate limits, PII exposure
Risk	API rate limit hit at ~50+ concurrent users; public PII
At 10x scale	Would need to migrate to PostgreSQL; Sheets API cannot handle it
At 100x scale	Sheets would fail completely — rate limit, concurrent writes, query latency

Decision 2: Streamlit as Full-Stack Framework

Aspect	Detail
Decision	Use Streamlit for both frontend rendering and data logic
Why	Fastest path to a functional web app in Python without frontend expertise
Alternative	Flask/Django + HTML templates; FastAPI + React
Tradeoff	Development speed vs. architectural flexibility and interview value
Risk	Cannot add REST API; hard to test UI; limited customization
At 10x scale	Streamlit's session model would cause memory pressure with concurrent users

Decision 3: `st.cache_data.clear()` on Every Load (Bug)

Aspect	Detail
Decision	FACT: Called <code>st.cache_data.clear()</code> on every page 2 load and on every search
Why (intended)	Wanted “real-time” data — feared stale cache
Problem	This defeats caching entirely — equivalent to having no cache at all
Correct approach	<code>@st.cache_data(ttl=300)</code> — cache for 5 minutes, auto-expire
Risk	At 50 concurrent users, this would hit Google Sheets’ 100 req/100s rate limit

Decision 4: Utility Functions in `utils.py`

Aspect	Detail
Decision	Extract <code>handle_nan</code> and <code>categorize_time</code> into a separate <code>utils.py</code>
Why	Good engineering instinct — separates pure logic from UI logic; enables unit testing
Strength	This is the most defensible engineering decision in the codebase
Tradeoff	Only 2 functions extracted; <code>fetch_data()</code> duplicated across pages (missed DRY opportunity)

Decision 5: GitHub Actions with 5 Workflows

Aspect	Detail
Decision	Set up 5 separate CI workflows
Why	Separation of concerns in CI — each job fails independently
Strength	Shows production CI thinking; useful for interview
Weakness	FACT: <code>continue-on-error: true</code> on lint and security makes them decorative
Fix	Remove <code>continue-on-error: true</code> ; require all checks to pass before merge

Decision 6: Column Access by Index (Magic Numbers)

Aspect	Detail
Decision	FACT: <code>usecols=[2, 3, 4, 5, 6, 7]</code> — columns accessed by integer index
Why	Quick and simple
Risk	Adding a column to the Google Sheet silently shifts all indices and breaks the app
Better approach	<code>usecols=['Name', 'Phone', 'Travel Time', 'Destination', 'Notes']</code> or a config constant

Decision 7: DevContainer Configuration

Aspect	Detail
Decision	FACT: <code>.devcontainer/devcontainer.json</code> configured for GitHub Codespaces
Why	Allows any contributor to start a dev environment in a browser instantly
Strength	Shows awareness of developer experience (DX)
Weakness	FACT: <code>--server.enableXsrfProtection false</code> in <code>postAttachCommand</code> — disables XSRF protection in dev

Decision 8: TruffleHog with @main (Unpinned)

Aspect	Detail
Decision	FACT: <code>uses: trufflesecurity/trufflehog@main</code>
Risk	Supply chain attack — if TruffleHog's main branch is compromised, malicious code runs in CI with full repo access
Fix	Pin to a specific commit SHA: <code>trufflesecurity/trufflehog@abc1234</code>
Interview value	Demonstrates security awareness if you proactively flag this

Decision 9: Public Google Sheet with PII

Aspect	Detail
Decision	FACT: Sheet URL is hardcoded and publicly accessible
Risk	Any person with the URL can bulk-download names and phone numbers of 1,100 students
Fix	Restrict Sheet to service account; never expose Sheet URL; use server-side filtering
Interview value	Proactively identifying this shows security maturity

Decision 10: Smoke Test with sleep 600

Aspect	Detail
Decision	FACT: <code>sleep 600</code> (10 minutes) hardcoded wait before health check
Why	Streamlit Cloud takes time to redeploy
Problem	Wastes 600 GitHub Actions minutes on every push; brittle timing
Better approach	Polling loop with exponential backoff: retry every 30s up to 15 minutes

7. End-to-End Data Flow

Feature: Search by Date

```

User selects date (e.g., 2024-10-14) and clicks "Search"
|
▼ [UI Layer – Streamlit reruns the page script]
date_input = st.date_input("Select a Travel Date:")
if st.button("Search"):
|
▼ [Validation – None currently; RECOMMENDATION: validate date is not in future]
|
▼ [Cache Clear – Bug]
st.cache_data.clear()
|
▼ [Data Fetch – External API Call]
fetch_data()
├─ conn = st.connection("gsheets", type=GSheetsConnection)
├─ conn.read(spreadsheet=url, usecols=[2,3,4,5,6,7])
  → HTTP GET to Google Sheets API
  → Returns: DataFrame with ~700 rows, 6 columns
|
▼ [Transformation – Pandas]
data['Travel Date'] = pd.to_datetime(data['Travel Date'], errors='coerce')
filtered_data = data[data['Travel Date'] == pd.to_datetime(date_input)]
|
├─ [If empty] → st.write("Sorry, No data found")
├─
└─ [If results found]
  |
  ▼ [Display – HTML card rendering]
  filtered_data.drop(columns=['Travel Date'])
  card_style() ← injects CSS
  for index, row in filtered_data.iterrows():
    st.markdown(f"""<div class="card">...(HTML with user data)...</div>""",
                unsafe_allow_html=True)
  |
  ▼ [Visualization – Matplotlib]
  categorize_time() applied to travel times
  plt.pie(sorted_distribution) → st.pyplot(plt)
  plt.pie(destination_distribution) → st.pyplot(plt)

```

Bottlenecks:

1. `st.cache_data.clear()` — full API call on every search
2. Full 700-row table download for every query (no server-side filter)
3. `filtered_data.iterrows()` — slow for large DataFrames (O(n) Python loop)
4. Two separate `plt.figure()` calls — synchronous, blocking

Failure Points:

1. Google Sheets API down → `try/except` catches it → `st.write(f"Error occurred: {e}")` (raw error shown to user)
2. Malformed date in Sheet → `errors='coerce'` converts to NaT silently (good defensive coding)
3. Column index shift → `KeyError` or wrong data displayed

Authorization: None — any user can search any date.

Logging: None — no record of what users searched, when, or what errors occurred.

Feature: All Days Summary

```

Page loads (no user interaction required)
|
▼
st.cache_data.clear() ← called on EVERY page load (bug: no user trigger)
|
▼
fetch_data() → conn.read(spreadsheet=url, usecols=[2, 4, 6])
|
▼
pd.to_datetime(data['Travel Date'], errors='coerce')
data['Travel Date'].dt.strftime('%d-%m-%Y')
|
▼
summary_dates = data['Travel Date'].value_counts()
summary_destinations = data['Destination'].value_counts()
|
▼
plt.pie(summary_dates) → st.pyplot()
plt.pie(summary_destinations) → st.pyplot()

```

Critical Bug: `st.cache_data.clear()` is called at module level (not inside a button handler), meaning it fires every time ANY user visits this page. This means every page load hits Google Sheets API with no caching benefit whatsoever.

8. Database Engineering Review**Schema (INFERRED from code)**

FACT: No schema documentation exists. Column names are inferred from comments in old commented-out code and from column indices.

Col Index	Column Name (inferred)	Data Type	Notes
0	Timestamp	datetime	Form submission time
1	Email	string	Google Form captures this
2	Name	string	User-provided
3	Phone	string	User-provided — PII
4	Travel Time	string	Time string (e.g., "2024-01-01 14:00:00")
5	Travel Date	string/date	Parsed to datetime in code
6	Destination	string	Free text
7	Notes/Message	string	Optional

Data Quality Issues

Issue	Current Handling	Better Handling
NaN values	<code>handle_nan()</code> → "Nil"	Same; already good
Invalid time strings	<code>categorize_time()</code> → "Invalid Time"	Log the bad entry
Date parsing errors	<code>errors='coerce'</code> → NaT silently	Alert on high NaT rate
Duplicate submissions	Not handled	Dedup by phone + date
Test entries	Not handled	Regex filter obvious test data

Read vs. Write Pattern

Pattern	Current
Read frequency	Every search, every summary page load
Write frequency	On every Google Form submission
Read-heavy vs. write-heavy	INFERENCE: Read-heavy (many searches vs. form submissions)
Transaction strategy	N/A — Google Sheets has no transaction support
Consistency	Eventual — Sheet API may lag form submissions

Database Interview Questions

Q: Why Google Sheets instead of a real database?

Strong: "Google Forms writes directly to Sheets — this gave me a zero-backend data ingestion pipeline. For 700 rows and low concurrent traffic, it was pragmatic. I've identified the scaling ceiling: Sheets API rate-limits at 100 req/100s. My migration plan is FastAPI + PostgreSQL."

Q: How would you migrate from Google Sheets to PostgreSQL?

Strong: "I'd write a migration script: read all rows from Sheets via the API, validate and transform them into the SQL schema, bulk-insert via `COPY` or batch inserts. I'd keep Sheets as the intake form temporarily by adding a webhook or scheduled sync job. Once migrated, I'd point the form to a FastAPI endpoint that writes directly to Postgres."

Q: What indexes would you add?

Strong: "Primary index on `id` (auto-increment). Composite index on `(travel_date, destination)` since every query filters on date first, then optionally on destination. The time column would benefit from a partial index on non-null values if we add match-by-time queries."

9. Security Audit

Current Security Posture

Threat	Current Protection	Risk Level
PII Exposure	None — Sheet is public	 Critical
XSS	Low — data from Sheet, not direct user input; but <code>unsafe_allow_html=True</code> used	 Medium
CSRF	Streamlit has partial protection (disabled in dev via <code>--server.enableXsrfProtection false</code>)	 Medium
SQL Injection	N/A — no SQL database	 N/A
NoSQL Injection	N/A — no NoSQL database	 N/A
SSRF	Low — only one outbound URL (hardcoded GSheets URL)	 Low
Replay attacks	N/A — no tokens or sessions	 Low
IDOR	N/A — no object IDs in app	 N/A
Privilege escalation	N/A — no auth at all	 Low
Dependency vulns	pip-audit in CI (weekly + on push)	 Medium
Secret leakage	TruffleHog in CI (but <code>@main</code> tag = supply chain risk)	 Medium
Supply chain	Unpinned <code>@main</code> on TruffleHog action	 Medium

Critical: PII Exposure Deep Dive

FACT: The Google Sheet URL is hardcoded in the source code as a public URL. The Sheet is publicly readable. This means:

1. Any person who finds the URL (or reads the source code on GitHub) can download all 1,100 user records
2. Names and phone numbers are accessible without any authentication
3. There is no audit trail of who has accessed the data

What a senior security engineer would criticize:

"This is a DPDP Act (India) / GDPR violation waiting to happen. You're storing user PII in a public spreadsheet, rendering it in a web app without masking, and have no consent mechanism beyond the Google Form's default disclosure. You need to: (1) restrict the Sheet to a service account, (2) mask phone numbers as +91-XXXX-12345 in the UI, (3) document your data retention policy, (4) add a privacy notice."

XSS Analysis

FACT: `unsafe_allow_html=True` is used in multiple places. The HTML template renders `handle_nan(row.iloc[0])` (Name) and `handle_nan(row.iloc[4])` (Message) directly into HTML.

```
st.markdown(f"""
    <div class="card">
        <p><strong> 📄 Message:</strong> {handle_nan(row.iloc[4])}</p>
    </div>
""", unsafe_allow_html=True)
```

Risk: If a user submits `<script>alert('xss')</script>` as their "Notes" in the Google Form, it would be rendered as executable JavaScript in every browser that views the search results.

Mitigation: `import html; html.escape(str(handle_nan(row.iloc[4])))` before rendering.

Secrets Management

FACT: The Google Sheets URL is hardcoded in source code. Service account credentials (if any) are managed via Streamlit Secrets. The actual secrets are not visible in the repository — this part is done correctly.

10. Failure Mode Analysis

"What breaks first?"

At current scale (~300 daily users at peak): Google Sheets API rate limit. At 10x (3,000 daily / ~30 concurrent): Streamlit Cloud's free tier memory limits.

Failure Mode Table

Failure	What Happens	User Impact	Current Mitigation	Better Mitigation
Google Sheets API down	<code>GSheetsConnection.read()</code> raises exception	Raw error shown: <code>st.write(f"Error occurred: {e}")</code>	bare <code>except</code> <code>Exception</code>	Circuit breaker; user-friendly fallback message; retry with backoff
Google Sheets API rate limit	429 error from API	Same as above	None	TTL caching (5 min); request queuing
Malformed data in Sheet	<code>pd.to_datetime(..., errors='coerce')</code> → NaT	Row silently dropped	<code>errors='coerce'</code> (partial)	Log bad rows; alert if >5% NaT rate
Streamlit Cloud goes to sleep	30–60 second cold start	User sees loading spinner	None	Uptime monitor (UptimeRobot ping every 5 min to keep alive)
GitHub Actions runner failure	CI skipped	No automated quality check	None	Retry: <code>runs-on: ubuntu-latest</code> restarts on transient failures
Google Form quota exceeded	Users can't submit	No new data	None	Monitor Google Workspace quotas
Cache cleared too aggressively	Every visit hits API	API rate limit approached	None (this IS the problem)	TTL-based cache
Column index shift in Sheet	Wrong data displayed or <code>KeyError</code>	Incorrect results silently	None	Named column access; schema

Failure	What Happens	User Impact	Current Mitigation	Better Mitigation
				validation on startup

11. Performance Engineering

Current Bottlenecks

Bottleneck	Location	Severity	Fix
Full table read per query	<code>conn.read(spreadsheet=url)</code>	● High	Server-side filtering OR TTL cache
Cache actively cleared	<code>st.cache_data.clear()</code> on every load	● High	Replace with <code>@st.cache_data(ttl=300)</code>
Synchronous chart rendering	<code>plt.figure()</code> + <code>st.pyplot()</code> × 2	● Medium	Move to Plotly (async-compatible)
<code>iterrows()</code> for card rendering	<code>for index, row in filtered_data.iterrows()</code>	● Medium	For 700 rows: acceptable. At 10k rows: use vectorized approach
No pagination	All matching cards rendered at once	● Medium	Limit to 20 per page with "Load more"

Caching Fix (Code)

```
# CURRENT (broken):
def fetch_data():
    conn = st.connection("gsheets", type=GSheetsConnection)
    return conn.read(spreadsheet=url, usecols=[2, 3, 4, 5, 6, 7])

if st.button("Search"):
    st.cache_data.clear() # ← destroys all cached data
    data = fetch_data()

# FIXED:
@st.cache_data(ttl=300) # Cache for 5 minutes
def fetch_data():
    conn = st.connection("gsheets", type=GSheetsConnection)
    return conn.read(spreadsheet=url, usecols=[2, 3, 4, 5, 6, 7])

if st.button("Search"):
    data = fetch_data() # Returns cached version if < 5 min old
```

Scale Behavior Estimates

Scale	Daily Users	Peak Concurrent	Behavior
Current	~100–300	~5–15	Works; caching bug not felt at this scale
10x	1,000–3,000	~50–150	Sheets API rate limit hit; Streamlit memory pressure
100x	10,000–30,000	~500–1,500	Streamlit Cloud would refuse connections; Sheets completely overwhelmed
1000x	100,000+	5,000+	Architecture must change entirely to FastAPI + DB + CDN

Migration Path for Scale

Current: Browser → Streamlit → Google Sheets API

10x Fix: Browser → Streamlit → `@st.cache_data(ttl=300)` → Google Sheets API (1 call per 5 min)

100x Fix: Browser → FastAPI → PostgreSQL (indexed queries, connection pool)
+ Background job: Google Forms webhook → FastAPI → PostgreSQL

1000x Fix: Browser → CDN → React SPA → FastAPI (load balanced) → PostgreSQL (read replica)
+ Redis cache layer + Celery async workers

12. Observability Review

Current State: NONE

FACT: Zero logging, zero metrics, zero error tracking, zero user analytics in the application code.

What Should Exist

Observability Layer	Tool	What to Track
Error tracking	Sentry	Exceptions with stack trace + context (date searched, user session)
Structured logging	Python <code>logging</code> module	Search events, API call durations, error rates
User analytics	Plausible / Umami	Page views, search frequency, popular dates
Uptime monitoring	UptimeRobot	Alert if app is down or cold-starting
API latency	Custom timing + Sentry	Time from button click to results rendered

How to Debug Without Observability (Current Reality)

Slow API question: "I can't tell — there's no timing instrumentation. I'd have to add `time.time()` before and after the `conn.read()` call and `st.write()` the result temporarily."

Production outage: "I'd check Streamlit Cloud logs, then manually test the Sheets API in a notebook. There's no alerting — I'd only know if a user told me."

Minimum Viable Observability (Code)

```
import logging
import time

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

def fetch_data():
    start = time.time()
    try:
        conn = st.connection("gsheets", type=GSheetsConnection)
        df = conn.read(spreadsheet=url, usecols=[2,3,4,5,6,7])
        logger.info(f"GSheets fetch: {len(df)} rows in {time.time()-start:.2f}s")
        return df
    except Exception as e:
        logger.error(f"GSheets fetch failed after {time.time()-start:.2f}s: {e}")
        raise
```

13. Testing Strategy Review

Current State

Test Type	Status	Count	Coverage
Unit tests	✅ Exists	7 tests	utils.py only
Integration tests	❌ Missing	0	—
E2E tests	❌ Missing	0	—
Contract tests	❌ Missing	0	—
Load tests	❌ Missing	0	—
Security tests	❌ Missing	0	—

FACT: 0% coverage of page logic (all 3 page files untested). Only `handle_nan` and `categorize_time` are tested.

Existing Tests (all in test_app.py)

```
test_handle_nan_with_real_value()      #  Good
test_handle_nan_with_nan()             #  Good
test_categorize_time_pm()              #  Good
test_categorize_time_am()              #  Good
test_categorize_time_noon()            #  Good
test_categorize_time_midnight()        #  Good
test_categorize_time_invalid()         #  Good
```

Missing Test Cases

Test Scenario	Why Missing	Priority
<code>categorize_time("2024-01-01 23:00:00")</code> → <code>"11PM - 12AM"</code>	Not tested — edge case	High
<code>categorize_time("2024-01-01 11:00:00")</code> → <code>"11AM - 12PM"</code>	Not tested — boundary	High
<code>handle_nan(0)</code> — should return 0, not "Nil"	Integer 0 is falsy; <code>isna(0) = False</code>	Medium
<code>handle_nan("")</code> — empty string	<code>pd.isna("") = False;</code> returns ""	Medium
<code>fetch_data()</code> with mocked GSheets connection	Integration test	High
Search with no results → correct message shown	UI logic test	High
Date parsing for various date formats	Data quality	Medium
Column index shift (col 3 renamed)	Regression test	Medium

Top 20 Test Scenarios

- 1. `categorize_time` — all 24 hours (parameterized)
- 2. `handle_nan` — None, `pd.NA`, `np.nan`, `float('nan')`, 0, "", False
- 3. `fetch_data` — mocked connection returns correct DataFrame shape
- 4. Date filter — correct rows returned for exact match
- 5. Date filter — empty result for date with no travelers
- 6. Date filter — date in future (validation test)
- 7. Date filter — malformed date in Sheet (coerce to NaT)

8. Card rendering — XSS payload in Name field → escaped in output
9. Card rendering — XSS payload in Message field → escaped in output
10. `card_style()` — returns valid CSS without syntax errors
11. Time distribution chart — empty series handled gracefully
12. Destination distribution — all “Nil” destinations handled
13. Summary page — correct total count across all dates
14. `summary_dates.value_counts()` — sorted correctly
15. `categorize_time` — “Invalid Time” returned for non-time string
16. `fetch_data` — GSheets API exception → raises and caught by try/except
17. Rate limit scenario — 429 from Sheets API → graceful error message
18. Column shift — accessing wrong column → assertion on shape
19. Concurrent searches — data isolation per session
20. `st.cache_data` TTL — second call within TTL returns cached result

Top 20 Edge Cases

1. User selects January 1 (default date picker start)
 2. Leap year date (Feb 29)
 3. Sheet has 0 rows
 4. Sheet has 10,000 rows — performance test
 5. All travelers on one date → all cards rendered
 6. Destination field empty for all rows on a date
 7. Travel time field empty for all rows on a date
 8. Name field contains HTML/JS injection
 9. Phone number contains +, -, spaces (display formatting)
 10. Duplicate rows (same person submitted twice)
 11. Sheet API returns empty DataFrame
 12. Network timeout during `conn.read()`
 13. `pd.to_datetime` with ambiguous date (01/02/03)
 14. Travel time at exactly 12:00:00 (noon)
 15. Travel time at exactly 00:00:00 (midnight)
 16. Destination with commas in name
 17. Sheet column count changes (fewer than expected)
 18. Unicode characters in Name (e.g., Hindi script)
 19. Very long message field (10,000 characters)
 20. Date input as string “None” or “null”
-

14. Deployment & DevOps

Build Process

FACT: No build step. Python doesn't require compilation. `pip install -r requirements.txt` is the only setup.

Release Process

```
Developer → git push → GitHub → Streamlit Community Cloud (auto-pull)
→ GitHub Actions (parallel):
  - ci.yml
  - lint.yml
  - security-audit.yml
→ smoke-test.yml (after 600s sleep)
```

Environment Strategy

Environment	Exists	Config
Development	INFERENCE: Local Streamlit run	<code>.streamlit/secrets.toml</code> locally
Staging	✗ Missing	None
Production	✓	Streamlit Community Cloud

Gap: No staging environment. Code goes directly from dev to production. Any bug pushed to main is immediately live.

Secrets Handling

FACT: Streamlit Secrets (`.streamlit/secrets.toml` locally; Streamlit Cloud UI for production) manages the Google service account credentials. The actual secret values are not in the repository.

Gap: The Google Sheet URL is hardcoded as a public URL in two files — this should be moved to a config constant or secret.

Rollback Strategy

Current: Git revert + push to main → Streamlit Cloud re-deploys. **No blue-green, no canary, no versioned releases.**

Docker Readiness

FACT: No Dockerfile exists. Not Docker-ready.

To Dockerize:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "main.py", "--server.address=0.0.0.0"]
```

CI/CD Maturity Assessment

Workflow	Purpose	Blocking?	Quality
ci.yml	pytest + coverage	✅ Blocking	Good
lint.yml	Ruff	❌ Not blocking (<code>continue-on-error: true</code>)	Cosmetic
security-audit.yml	pip-audit + TruffleHog	❌ Not blocking	Cosmetic
smoke-test.yml	HTTP health check	✅ Blocking	Fragile (sleep 600)
update-docs.yml	Auto-PR	N/A	Generates hardcoded content

Maturity Score: 5/10 — 5 workflows is impressive for a student; execution quality needs polish.

15. Code Quality Review

SOLID Analysis

Principle	Status	Notes
S — Single Responsibility	Partial	<code>utils.py</code> is clean. Page files mix data fetching, transformation, and rendering.
O — Open/Closed	N/A	No class hierarchy to evaluate
L — Liskov Substitution	N/A	No inheritance
I — Interface Segregation	N/A	No interfaces
D — Dependency Inversion	✗	Page files directly instantiate GSheets connection; not injectable

DRY Violations

Violation	Files	Fix
<code>SHEET_URL</code> hardcoded twice	<code>Search_by_Date.py</code> , <code>All_Days_Summary.py</code>	<code>config.py: SHEET_URL = "..."</code>
<code>fetch_data()</code> defined twice	Both page files	Move to <code>utils.py</code>
Column index magic numbers	Both page files	Named constants in <code>config.py</code>

Code Smells

Smell	Location	Severity
100+ lines of commented-out dead code	All page files	 High
Bare <code>except Exception as e</code>	<code>Search_by_Date.py</code>	 Medium
<code>st.write(f"Error occurred: {e}")</code> — raw exception to user	Same	 Medium
<code>st.cache_data.clear()</code> at module level	<code>All_Days_Summary.py</code>	 High
No type hints	All files	 Medium
No docstrings on page-level functions	All page files	 Low

What a Staff Engineer Would Improve

```
# 1. Create config.py
SHEET_URL = "https://docs.google.com/spreadsheets/d/..."
FETCH_COLUMNS = ['Name', 'Phone', 'Travel Time', 'Travel Date', 'Destination', 'Notes']
CACHE_TTL_SECONDS = 300

# 2. Fix fetch_data in utils.py (shared, cached, typed)
@st.cache_data(ttl=CACHE_TTL_SECONDS)
def fetch_travel_data() -> pd.DataFrame:
    """Fetch all travel records from Google Sheets with TTL caching."""
    conn = st.connection("gsheets", type=GSheetsConnection)
    return conn.read(spreadsheet=SHEET_URL, usecols=list(range(2, 8)))

# 3. Type hints on all functions
def handle_nan(value: object) -> object:
    """Return 'Nil' if value is NaN, otherwise return value unchanged."""
    return "Nil" if pd.isna(value) else value

def categorize_time(time_str: str) -> tuple[str, int]:
    """Categorize a datetime string into a 1-hour AM/PM interval."""
    ...

# 4. Sanitize before HTML rendering
import html
safe_message = html.escape(str(handle_nan(row.iloc[4])))

# 5. Specific exception handling
try:
    data = fetch_travel_data()
except ConnectionError as e:
    logger.error(f"GSheets connection failed: {e}")
    st.error("Unable to fetch data. Please try again in a moment.")
    st.stop()
except Exception as e:
    logger.error(f"Unexpected error: {e}", exc_info=True)
    st.error("An unexpected error occurred. The team has been notified.")
    st.stop()
```

16. Production Readiness Scorecard

Dimension	Score	Justification	Biggest Weakness	Fastest Improvement
Architecture	3/10	No API layer, tightly coupled, no separation	No REST API — can't extend	Create <code>utils.py</code> as service layer
Security	2/10	PII publicly exposed; unsafe HTML; no auth	Public PII in Sheet	Mask phone numbers immediately
Testing	4/10	7 unit tests, CI runs, coverage artifact	0% page coverage; no mocks	Add mocked GSheets tests
Scalability	2/10	Full table reads; cache defeated; no pagination	Cache bug at any scale	Fix TTL caching
Reliability	4/10	Live, working app; Streamlit Cloud uptime OK	No failover, no retry	Add retry logic + graceful errors
Maintainability	2/10	Dead code, magic numbers, duplicated logic	Dead code everywhere	Delete commented code
Performance	2/10	Cache actively cleared; synchronous blocking	API called on every load	TTL cache fix
Observability	1/10	Zero logging, metrics, alerting	No visibility at all	Add Python logging + Sentry
Deployment maturity	5/10	5 CI workflows, live deployment	CI not blocking; no staging	Remove <code>continue-on-error</code>

Overall Production Readiness: 3/10

This is a real product that works at current scale. It is not production-ready by enterprise standards. The CI/CD pipeline and real user base are the strongest signals of engineering intent.

17. Staff Engineer Review

What Is Impressive

- **FACT:** 1,100+ real users — this is a production system, not a toy
- **FACT:** 5 GitHub Actions workflows including security scanning and smoke testing
- **FACT:** Separation of utility functions into testable `utils.py`
- **FACT:** 7 unit tests with CI coverage upload
- **FACT:** DevContainer configured for instant developer onboarding
- **FACT:** TruffleHog secret scanning — most student projects skip this entirely
- **FACT:** Dependency vulnerability scanning with pip-audit
- **INFERENCE:** The app genuinely solved a real problem (measurable by user adoption)

What Feels Junior

- **FACT:** 50%+ of file content is commented-out old code — no discipline around dead code removal
- **FACT:** `st.cache_data.clear()` on every load — fundamentally misunderstands caching
- **FACT:** `continue-on-error: true` on security and lint — CI is theater, not quality gate
- **FACT:** Column indices as magic numbers — `usecols=[2,3,4,5,6,7]` with no constants
- **FACT:** `st.write(f"Error occurred: {e}")` — exposing raw exception text to users
- **FACT:** No type hints anywhere
- **INFERENCE:** No understanding that `iterrows()` is slow (though acceptable at this scale)

What Feels Intermediate

- Extracting utility functions to a separate testable module
- Writing parameterized unit tests for time categorization
- Setting up multiple CI workflows
- Custom CSS card components with hover effects

What Feels Senior (if added)

- Identifying the PII exposure risk proactively
- Proposing the TTL-based caching fix with reasoning
- Planning the FastAPI + PostgreSQL migration path
- Pinning GitHub Actions to commit SHAs for supply chain security

What Blocks Production Approval

1. PII exposure (Critical)
2. No authentication (Critical)
3. Caching bug (High)

4. No staging environment (High)
5. CI not actually blocking merges (Medium)
6. Raw exception messages to users (Medium)

Technical Debt Inventory

Debt	Files	Effort to Fix
Dead code	All page files	30 min
Caching bug	Both page files + Summary	30 min
DRY violation (fetch_data, SHEET_URL)	Both page files	1 hr
Magic number column indices	Both page files	1 hr
No type hints	All files	2 hr
No input validation	Search page	1 hr
PII masking	Search page	1 hr
CI not blocking	lint.yml, security-audit.yml	5 min

18. Project Learnings

Technical Skills Demonstrated

Skill	Evidence
Python web development	Streamlit multi-page app
Data processing	Pandas NaN handling, datetime parsing, filtering
Data visualization	Matplotlib pie charts with custom sorting
External API integration	Google Sheets via st-gsheets-connection
CI/CD pipeline design	5 GitHub Actions workflows
Security tooling	pip-audit + TruffleHog
Unit testing	pytest with coverage
HTML/CSS	Custom card components with hover effects
Developer tooling	Ruff linter, DevContainer

Engineering Principles Learned (Demonstrable)

- **Separation of concerns:** `utils.py` as pure utility module
- **Testability:** Functions designed to be unit-testable without Streamlit
- **CI as quality gate** (even if imperfectly executed)
- **Real-world data messiness:** NaN handling, type coercion, string parsing

Tradeoff Thinking Demonstrable

- "I chose Google Sheets over a real DB because the ingestion pipeline (Forms → Sheets) was zero cost and zero code. I accepted the tradeoff of no indexing and API rate limits, which I've now identified as a scaling risk."
- "I used Streamlit for speed of delivery over architectural elegance. With 1,100 users gained, I'd say that tradeoff paid off for v1."

What Should Be Redesigned Today

1. **Caching strategy** — fix the `clear()` bug immediately
2. **PII masking** — ethical and legal obligation
3. **Error handling** — specific exceptions, user-friendly messages
4. **Dead code removal** — before any interview code review

19. Resume Claim Validation

Bullet 1: “Engineered a high-traffic carpool hub serving 300+ daily requests at peak”

Aspect	Status
Evidence	FACT: README states “300+ daily requests at peak times”
Defensible?	Yes, if you can explain what “request” means in Streamlit
Follow-up	“How did you measure 300+ daily requests?”
Strong defense	“A ‘request’ in Streamlit terms is a page load + any button click triggering a rerun. I tracked this through Streamlit Cloud’s built-in dashboard analytics which shows viewer counts. At Diwali peak, I saw 300+ viewers per day.”
Weak answer	“I just estimated it.” ← Immediately destroys credibility

Bullet 2: “1,100+ users”

Aspect	Status
Evidence	FACT: README states “Trusted by 1000+ Users along with 700+ entries”
Defensible?	Yes — 700 form entries is a concrete, verifiable number
Follow-up	“How do you define a ‘user’ — unique visitors or form submitters?”
Strong defense	“700+ unique form submitters. Unique visitors (including people who search without submitting) were higher — I estimate 1,100+ based on Streamlit’s viewer analytics and the fact that each form entry typically generates 2–4 searches by potential carpool partners.”

Bullet 3: "90% automated logistics"

Aspect	Status
Evidence	INFERENCE
Defensible?	Weak — vague claim
Follow-up	"What does 90% automated mean? What's the 10% that's manual?"
Strong defense	"Data ingestion (Google Forms → Sheets) is fully automated. The matchmaking discovery — searching and choosing a partner — is self-service by users. I'd rephrase this to: 'zero-ops data ingestion pipeline via Google Forms + Sheets API'."
Recommendation	Remove this bullet or rephrase with specifics

Bullet 4: "30% faster commute"

Aspect	Status
Evidence	None in codebase
Defensible?	Very weak — no measurement methodology
Follow-up	"How did you measure a 30% faster commute?"
Recommendation	Remove this bullet entirely unless you have survey data

Strong Replacement Resume Bullets

- Built V_Carpool, a Python + Streamlit travel matchmaking platform adopted by **1,100+** university students with **700+** form submissions at peak usage
- Designed a serverless data pipeline: Google Forms → Google Sheets → Streamlit dashboard eliminating all backend infrastructure **for** data ingestion
- Implemented **5** GitHub Actions CI/CD workflows: automated **testing** (pytest + coverage upload), **Ruff** linting, dependency CVE **scanning** (pip-audit), secret **scanning** (TruffleHog), and post-deploy HTTP smoke testing
- Authored **7** unit tests **for** core data-processing **utilities** (NaN handling, time categorization) achieving CI-enforced coverage reporting on every commit
- Identified and documented a critical PII exposure **risk** (**public** Google Sheet containing **1,100** user phone numbers) and proposed a remediation plan involving service account access control and UI-level phone masking

20. Interview Story Generator

STAR: Architecture Decision

Situation: I needed a data store for a travel matchmaking app. I had no backend infrastructure and limited time before the Diwali holiday window.

Task: Collect traveler preferences from 500+ students and make them searchable by date within 48 hours.

Action: Instead of setting up a database, I used Google Forms as the data intake layer (it writes directly to Google Sheets). I built a Streamlit app that reads the Sheet in real-time. This gave me a complete data pipeline with zero backend code and zero ops overhead.

Result: Deployed in 2 days. 1,100+ users adopted it. 700+ entries collected.

What I'd do differently: I'd still use the same intake pipeline (Forms → Sheets), but I'd add server-side caching with a TTL, restrict the Sheet to a service account, and build a FastAPI backend to decouple data access from the UI.

STAR: Performance Optimization

Situation: I noticed that every time a user visited the "All Days Summary" page, the app was making a fresh API call to Google Sheets — even if the data hadn't changed in the last 5 minutes.

Task: Reduce unnecessary API calls without serving stale data.

Action: I identified that `st.cache_data.clear()` was being called at module level on page load, which defeats Streamlit's caching mechanism entirely. The correct fix is `@st.cache_data(ttl=300)` on the `fetch_data()` function — this caches the data for 5 minutes and auto-expires.

Result: (RECOMMENDATION — not yet implemented) This would reduce Google Sheets API calls by ~90% during normal usage, keeping the app well within the 100 req/100s rate limit even under load.

STAR: Security Consideration

Situation: During a code review of my own project, I realized the Google Sheet URL was hardcoded in the source code as a public URL, and the Sheet itself had public read access enabled.

Task: Assess the risk and propose a remediation.

Action: I documented the threat: anyone with the URL (visible on GitHub) could bulk-download 1,100 students' names and phone numbers without authentication. I also identified an XSS risk: user-submitted data in the "Notes" field is rendered directly as HTML via `unsafe_allow_html=True` without sanitization.

Result: (RECOMMENDATION) Remediation plan: restrict Sheet to service account only, add `html.escape()` before rendering user data, mask phone numbers in UI to format `+91-XXXXX-12345`.

STAR: Difficult Bug Fixed

Situation: Users reported that the search results felt slow and the app seemed to always be fetching fresh data rather than loading quickly.

Task: Investigate and fix the perceived performance issue.

Action: I traced the code and found that `st.cache_data.clear()` was being called inside the search button handler — and also at module level on the summary page. This completely defeats Streamlit's built-in caching, meaning every search and every page visit triggers a full Google Sheets API call, even for data that hadn't changed.

Result: (RECOMMENDATION) Replacing with `@st.cache_data(ttl=300)` would fix this. The root cause was a misunderstanding of Streamlit's caching model — I thought clearing the cache ensured “freshness,” but the correct approach is TTL-based expiry.

21. Mock Interview Drill

15 Basic Questions

#	Question	Ideal Answer	Weak Answer	Why Weak Fails
1	What does this app do?	"Travel matchmaking app — students submit date/time/destination via form, others search by date to find travel partners."	"It's a carpool app."	Too vague; no specifics
2	What language is it written in?	"Python with Streamlit for the web framework, Pandas for data processing, Matplotlib for charts."	"Python."	Misses the stack
3	How is data stored?	"Google Sheets, fed by Google Forms submissions via the GSheets API connector."	"In a database."	Wrong; shows no understanding
4	How many users have used it?	"700+ form submitters; 1,100+ total visitors including searchers. Peak: 300+ daily requests at Diwali."	"About 1,000."	No specifics; can't be grilled further
5	How do users submit data?	"Via an external Google Form that writes directly to Google Sheets — no backend code needed for ingestion."	"They fill a form."	Misses the architecture insight
6	What is Streamlit?	"A Python framework that renders web UIs from pure Python. No HTML/JS required. It reruns the script on every interaction."	"A web framework."	Too vague
7	How is the app deployed?	"Streamlit Community Cloud — it auto-deploys from GitHub on every push to main."	"On the cloud."	No specifics
8	What tests do you have?	"7 unit tests in test_app.py covering handle_nan and categorize_time. Run in CI via pytest with coverage."	"Some unit tests."	Can't be defended with specifics
9	What is CI/CD?	"Automated pipelines that run on every code push. I have 5 workflows: testing, linting, security scanning, smoke testing, doc updates."	"It's automated testing."	Incomplete; misses CD
10	What does pytest-cov do?	"Measures what percentage of code is executed during tests and	"It runs tests."	Wrong tool description

#	Question	Ideal Answer	Weak Answer	Why Weak Fails
		generates an HTML coverage report."		
11	What is Ruff?	"A very fast Python linter and formatter written in Rust. Replaces flake8, isort, and black with one tool."	"A linter."	Misses the key differentiator (Rust-speed)
12	What is pip-audit?	"Scans installed Python packages against known CVE databases to detect vulnerable dependencies."	"It checks packages."	Too vague
13	What is TruffleHog?	"Secret scanner that searches git history for accidentally committed API keys, passwords, tokens."	"It's a security tool."	No specifics
14	What is a DevContainer?	"A configuration file that defines a Docker-based development environment for VSCode/Codespaces — anyone can spin up the dev env in one click."	"It's for development."	No specifics
15	What are the pages in the app?	"Home (navigation), Search by Date (core feature), All Days Summary (analytics overview), Credits (developer info)."	"Three or four pages."	Can't describe content

15 Intermediate Questions

#	Question	Ideal Answer	Follow-Up
1	Why did you use Google Sheets as a database?	"Zero-backend data ingestion via Forms → Sheets. Pragmatic for v1 at this scale. I know the tradeoffs: no indexing, 100 req/100s rate limit, PII risk."	"What's the migration path?"
2	How does caching work in Streamlit?	" <code>@st.cache_data</code> caches function return values; invalidated by TTL or <code>cache_data.clear()</code> . My current code has a bug — I call <code>clear()</code> on every load, defeating caching."	"What's the correct fix?"
3	What is the purpose of <code>utils.py</code> ?	"Single Responsibility: pure utility functions isolated from UI logic. This makes them independently testable without Streamlit running."	"Why not put the logic in the page file?"
4	How does Streamlit routing work?	"Files in the <code>pages/</code> directory are automatically registered as routes. Naming convention: <code>{order}_{emoji}_{Name}.py</code> ."	"How do you pass data between pages?"
5	What does <code>errors='coerce'</code> do in Pandas?	"When <code>pd.to_datetime()</code> encounters an unparseable value, instead of raising an exception it converts it to NaT (Not a Time)."	"What's the risk of silent NaT conversion?"
6	Why is <code>unsafe_allow_html=True</code> risky?	"It allows arbitrary HTML in Streamlit — if user-submitted data is rendered without sanitization, it's an XSS vector."	"How would you fix it?"
7	How does <code>iterrows()</code> work?	"Iterates over DataFrame rows returning (index, Series) tuples. It's slow — O(n) Python-level iteration vs vectorized C operations."	"What's a faster alternative?"
8	Explain <code>value_counts()</code>	"Returns a Series of value frequencies in descending order. Used here to count travelers per date and per destination."	"What's the time complexity?"
9	What is a GitHub Actions workflow?	"A YAML file in <code>.github/workflows/</code> defining automated jobs triggered by events (push, PR, schedule). Jobs run on GitHub-managed VMs."	"How do you share data between jobs?"
10	What does <code>continue-on-error: true</code> do?	"Allows a step to fail without failing the entire job. In my lint and security workflows, this	"Should you fix this?"

#	Question	Ideal Answer	Follow-Up
		means failures are logged but don't block the build — making them decorative."	
11	How does the smoke test work?	"After deploy, it waits 600 seconds (hardcoded), then curls the live URL and checks for HTTP 200. Also checks page content for error keywords."	"What's wrong with <code>sleep 600</code> ?"
12	What is <code>pd.isna()</code> ?	"Returns True for NaN, None, <code>pd.NA</code> , and <code>np.nan</code> . Used in <code>handle_nan()</code> to replace missing values with 'Nil'."	"Does it return True for empty string?"
13	What are the columns read from Google Sheets?	"Columns at indices 2–7: Name, Phone, Travel Time, Travel Date, Destination, Notes. Accessed by integer index."	"What's the risk of integer indexing?"
14	How does <code>categorize_time()</code> work?	"Parses a datetime string, extracts the hour, and returns an interval string like '2PM - 3PM' plus the raw hour for sorting."	"What happens at hour 23 (11PM)?"
15	What does the smoke test check that curl doesn't?	"Content validation: searches the HTML for 'error', 'exception', 'traceback' keywords. HTTP 200 means the page loaded; content check means it's not just an error page."	"Could a broken app still return 200?"

15 Advanced Questions

#	Question	Ideal Answer
1	How would you fix the caching bug?	" <code>@st.cache_data(ttl=300)</code> on <code>fetch_data()</code> . Remove all <code>st.cache_data.clear()</code> calls. TTL ensures freshness within 5 min while serving cached data between calls."
2	How would you add authentication?	"Streamlit-authenticator library for simple cases; or redirect to a Google OAuth flow and validate the token server-side before rendering any data."
3	How would you migrate to FastAPI + PostgreSQL?	"Step 1: Create FastAPI app with <code>/api/travelers?date=YYYY-MM-DD</code> endpoint. Step 2: Migrate Sheets data via script to PostgreSQL. Step 3: Add webhook or scheduled job to sync new Form submissions. Step 4: Update Streamlit to call internal API instead of Sheets directly."
4	What indexes would you add to the PostgreSQL schema?	"Composite index on <code>(travel_date, destination)</code> for date+destination filter queries. Index on <code>travel_date</code> alone for date-only searches. Partial index on <code>phone IS NOT NULL</code> for matching queries."
5	How would you implement rate limiting?	"Streamlit session state to track search counts: <code>st.session_state.search_count</code> . Increment on each search; show error if $> 10/\text{minute}$. For real rate limiting, put NGINX or a reverse proxy in front."
6	How would you prevent XSS?	" <code>import html; html.escape(str(value))</code> before any user-provided string goes into an <code>unsafe_allow_html=True</code> block. Or switch to Streamlit native components that don't allow raw HTML."
7	How would you handle the Google Sheets API rate limit?	"TTL caching is the primary mitigation. At 10x scale: move to PostgreSQL with a background sync job from Sheets. Implement exponential backoff retry in <code>fetch_data()</code> ."
8	What is a supply chain attack?	"An attacker compromises a dependency (e.g., a GitHub Action) and injects malicious code that runs in your CI pipeline. Pinning to commit SHAs mitigates this because the SHA is content-addressed — it won't change even if the repo is compromised."
9	Explain Streamlit's execution model	"Every user interaction (button click, slider, input change) reruns the entire Python script from top to bottom in a

#	Question	Ideal Answer
		new execution context. State between interactions is managed via <code>st.session_state</code> . This is different from traditional request-response — it's more like a reactive framework."
10	How does <code>@st.cache_data</code> work internally?	"It serializes function arguments to a cache key, checks if a cached result exists and isn't expired (TTL), returns cached result if hit. Uses pickle for serialization. Per-session by default but can be shared across sessions."
11	What is the difference between <code>st.cache_data</code> and <code>st.cache_resource</code> ?	" <code>cache_data</code> — for data objects (DataFrames, lists); creates a copy per call. <code>cache_resource</code> — for shared resources (DB connections, ML models); returns same object across all sessions."
12	How would you implement AI matchmaking?	"Score each traveler against the searching user: $\text{destination_match} * 10 + \text{time_proximity_score} * 10 + \text{partial_name_match} * 2$. Time proximity: $\text{abs}(\text{user_hour} - \text{traveler_hour}) \leq 1 = \text{full score}$. Sort by score descending. Advanced: TF-IDF vectorize destinations, cosine similarity."
13	Explain <code>pd.to_datetime(time_str).hour</code>	"Parses an ISO datetime string to a Pandas Timestamp object, then extracts the hour attribute (0–23). If parsing fails and <code>errors='raise'</code> (default), raises <code>ParserError</code> ."
14	What is <code>plt.clf()</code> and why is it called?	"Clears the current Matplotlib figure. Without it, subsequent <code>plt.pie()</code> calls would add to the same figure object, overlapping charts. In Streamlit, each <code>st.pyplot()</code> call should receive a fresh figure."
15	How does TruffleHog detect secrets?	"Scans git commit history (not just current code) using regex patterns and entropy analysis. Entropy analysis finds high-randomness strings that are likely to be keys/tokens. <code>--only-verified</code> only reports secrets it can confirm are live."

15 System Design Questions

#	Question	Ideal Answer
1	Design the matchmaking algorithm at scale	"Read user's date/time/destination. Score all travelers on that date: exact destination = 10pts, time within 1hr = 10pts, time within 2hr = 5pts. Rank top 10. Cache scored results per date for 5 minutes. At scale: precompute score matrix nightly via batch job."
2	How would you handle 10,000 concurrent users?	"Move to FastAPI + PostgreSQL with connection pooling (pgBouncer). Add Redis cache layer (TTL=5min per date key). Put CDN in front of static assets. Horizontal scale FastAPI behind NGINX load balancer."
3	Design the notification system	"After match found, send WhatsApp/SMS via Twilio API. Store notification preference in DB. Use Celery + Redis queue for async delivery. Retry failed notifications with exponential backoff."
4	How would you make this real-time?	"WebSocket connection from browser to server. When new form submission arrives (webhook from Google Forms), push update to all connected clients viewing the same date. FastAPI + WebSockets for this."
5	Design the auth system	"OAuth2 with Google (university SSO). JWT access token (15min TTL) + refresh token (7 days). Store refresh tokens in Redis with user_id as key. Validate token on every API call. Role table: student, admin."
6	How would you add bidirectional filtering (campus→home AND home→campus)?	"Add 'direction' field to the data schema. Current app is one-directional (campus→home). Add a radio button to the search form: 'Going home' / 'Returning to campus'. Filter on both date AND direction."
7	Design the admin panel	"Password-protected page (Streamlit secrets). Shows: total users, daily search volume, error rate, top dates, top destinations. Ability to delete/flag entries. Audit log of admin actions."
8	How would you add in-app messaging?	"Store messages in PostgreSQL. API: <code>POST /messages</code> (sender_id, recipient_id, text). WebSocket for real-time delivery. Notification badge count in UI."
9	How would you implement data export?	" <code>st.download_button(label='Download CSV', data=df.to_csv(index=False), file_name='results.csv', mime='text/csv')</code> — 10 lines of code. For large datasets, stream the response."
10	How would you do load testing?	"Locust or k6. Define user scenarios: visit home, visit search page, click search (with a test date). Ramp from 1 to 1000 virtual users. Observe: response time, error rate, GSheets API rate limit breach point."
11	How would you handle GDPR-style data	"Add 'Delete my data' button. User provides their phone number. System searches Sheet for matching entries and deletes rows (requires

#	Question	Ideal Answer
	deletion?	write permissions via service account). Log the deletion request."
12	How would you build the demand forecasting model?	"Time series: Prophet model on daily entry counts. Features: day_of_week, is_holiday, days_before_semester_end. Train on historical 700+ entries. Predict next 30 days. Serve via FastAPI <code>/api/forecast</code> endpoint. Visualize in Streamlit."
13	Design for 99.9% uptime	"Blue-green deployment: two identical environments, switch traffic after smoke test. Database: Postgres primary + read replica with automatic failover. CDN for static assets. Circuit breaker on GSheets API. Uptime SLA monitoring."
14	How would you shard the database?	"At current scale: not needed. At 1M entries: shard by travel_date (time-based sharding). Older dates become cold storage (S3). Recent dates stay in hot PostgreSQL. Query router determines which shard to hit."
15	How would you implement Disaster Recovery?	"Daily database backups to S3 (automated via pg_dump). RTO: 2 hours (restore from backup). RPO: 24 hours (max data loss). For better RPO: continuous WAL archiving. Runbook documenting restore procedure."

22. Final Project Mastery Cheatsheet

2-Minute Explanation

"V_Carpool is a Python + Streamlit web app I built to solve a real problem: hundreds of university students traveling on the same dates with no way to find each other. I used Google Forms for data intake — it writes directly to Google Sheets — and built a Streamlit app on top that lets students search by date, see who's going where, and reach out directly. It's served 1,100+ real users with 700+ form entries. I also set up 5 GitHub Actions CI/CD pipelines: automated testing with coverage reporting, Ruff linting, dependency vulnerability scanning with pip-audit, secret scanning with TruffleHog, and HTTP smoke testing after each deploy."

5-Minute Explanation (adds architecture depth)

[Use the 2-minute explanation, then add:] "The architecture is deliberately simple but has real engineering decisions behind it. The data ingestion pipeline uses Google Forms → Sheets — no backend code required, which let me launch in 48 hours. The app reads from Sheets using the `st-gsheets-connection` library and uses Pandas for filtering and data cleaning."

I extracted pure utility functions — NaN handling and time categorization — into a separate `utils.py` so they're independently testable. The 7 unit tests cover all time-of-day cases including midnight and noon boundary conditions.

The biggest technical challenge I've identified is a caching bug: I was calling `st.cache_data.clear()` on every page load, which defeats Streamlit's caching entirely. The correct fix is TTL-based caching with `@st.cache_data(ttl=300)`.

The main security concern I've found is that the Google Sheet is publicly accessible — exposing 1,100 users' names and phone numbers. My remediation plan is to restrict it to a service account, add phone masking in the UI, and sanitize user inputs before HTML rendering."

10-Minute Deep Dive

[Use the 5-minute explanation, then add:] "Let me walk through the end-to-end flow for a search. User selects a date and clicks Search. Streamlit reruns the page script, calls `fetch_data()` which hits the Google Sheets API and returns a ~700-row DataFrame. Pandas filters it to rows matching the selected date. For each matching row, we render an HTML card using `st.markdown(unsafe_allow_html=True)`. Then we call `categorize_time()` on each travel time string to bin them into 1-hour intervals, and render two pie charts — one for time distribution, one for destination distribution.

The CI pipeline has 5 workflows. The test pipeline runs pytest with `--cov` and uploads the HTML coverage report as a GitHub Actions artifact. The security pipeline runs pip-audit weekly to check for CVEs and runs TruffleHog on the git history to detect leaked secrets. The smoke test pings the live URL after deploy to verify HTTP 200 and absence of error text.

If I were scaling this to 10x users, the first thing to fix is the caching bug — that alone would reduce API calls by 90%. The second is migrating from Google Sheets to PostgreSQL with a FastAPI backend, adding a `(travel_date, destination)` composite index. At 100x, I'd add a Redis cache layer, horizontal scale FastAPI behind NGINX, and implement real authentication."

Google Forms → Google Sheets → Streamlit App → User Browser

```

      ↑           ↑
(data intake)  (st-gsheets-connection)
               (Pandas processing)
               (Matplotlib charts)
               (Custom CSS cards)

```

Risk	Status	Fix
PII exposed	🔴 Open	Restrict Sheet; mask phones
XSS	🟡 Low-risk	<code>html.escape()</code> all user data
Supply chain	🟡 Open	Pin GH Actions to SHA
Secret leakage	✅ Managed	Streamlit Secrets; TruffleHog CI
Auth	🔴 None	Google OAuth / Streamlit-auth

- 7 unit tests; pytest + coverage in CI
- Tests cover `utils.py` only; 0% page coverage
- Missing: mocked GSheets, E2E, integration, security tests

- Current ceiling: ~50 concurrent users (Sheets rate limit)
- Fix at 10x: TTL caching + PostgreSQL migration
- Fix at 100x: FastAPI + Redis + horizontal scaling

- Push to main → Streamlit Cloud auto-redeploys
- GitHub Actions: 5 workflows, 2 actually blocking builds
- No staging environment; no Docker; no rollback strategy beyond git revert

Top 20 Talking Points

1. 1,100+ real users — production, not a demo
2. 5 GitHub Actions CI/CD workflows
3. TruffleHog secret scanning in CI
4. pip-audit CVE scanning in CI
5. Identified and documented PII exposure risk
6. Utility functions in testable `utils.py`
7. 7 unit tests with CI coverage reporting
8. TTL caching bug identified — can explain the correct fix
9. DevContainer for instant onboarding
10. Google Forms → Sheets = zero-backend ingestion pipeline
11. Smoke test against live URL post-deploy
12. Ruff for fast linting + formatting
13. Custom CSS card components with hover animation
14. Matplotlib pie charts with custom time-slot sorting
15. Pandas datetime parsing with `errors='coerce'`
16. Multi-page Streamlit routing via `pages/` directory
17. Unpinned `@main` TruffleHog action — identified supply chain risk
18. Column index magic numbers — identified as fragility
19. Proposed FastAPI + PostgreSQL migration path
20. Real-world problem solved with measurable adoption

Top 20 Interviewer Traps

1. "How did you measure 300+ daily requests?" → Have a specific answer ready
2. "What's the database schema?" → Explain columns; admit no formal schema docs
3. "Why does `cache_data.clear()` matter?" → Explain the bug in detail
4. "Is the Google Sheet private?" → Honest: no. Explain the risk and fix.
5. "Can your app handle 10,000 users?" → No. Explain the ceiling and migration plan.
6. "What happens if Google Sheets is down?" → Bare except catches it; raw error shown to user
7. "Are your CI checks actually blocking?" → Honest: lint and security use `continue-on-error`
8. "What does `unsafe_allow_html=True` risk?" → XSS; explain the sanitization fix
9. "What would you redesign from scratch?" → Caching, PII handling, auth, FastAPI migration
10. "Why `iterrows()` and not a vectorized operation?" → Honest: performance acceptable at 700 rows; know the alternative
11. "What's the XSS attack vector?" → Notes field via Google Form → rendered without escaping
12. "Why is `@main` on TruffleHog a problem?" → Supply chain attack; explain SHA pinning
13. "How do you know column 4 is 'Travel Date'?" → Inferred from commented code; admit fragility
14. "What's your staging environment?" → Honest: none. Explain the risk.
15. "What's your rollback strategy?" → Git revert + push. Not ideal.

16. "What does `pd.isna()` return for empty string?" → False. Empty string is not NaN.
17. "What breaks when you add a column to the Sheet?" → Column index shift; all integer indices wrong
18. "Why does `sleep 600` make the smoke test bad?" → Wastes CI minutes; brittle timing
19. "Is pytest-cov enforcing a coverage threshold?" → No. It reports but doesn't block. Explain the fix.
20. "What's the difference between `cache_data` and `cache_resource`?" → Data copies vs. shared resource instances

23. Feature Deep Dive Cross-Examination

Feature 1: Search by Date

Business Purpose: Core feature — lets a user discover who else is traveling on their chosen date.

User Value: Eliminates manual WhatsApp coordination; shows all travelers on a date in 1 click.

Frontend: Date picker (`st.date_input`), Search button, HTML card components, Matplotlib charts

Backend: `fetch_data()` → Google Sheets API → Pandas filter → rendered HTML

Database Interactions: Full table read; Python-side filtering

Security: Renders user-submitted data in HTML without sanitization (XSS risk)

Performance: Full 700-row API call per search; no pagination; synchronous blocking charts

Level 1: Basic Questions

Q: What does the Search by Date page do?

Strong: "It lets users pick a travel date and see all registered travelers for that date, displayed as cards with name, phone, travel time, destination, and notes. It also shows two pie charts: travel time distribution and destination distribution." **Weak:** "It searches for carpools." → Too vague; no specifics.

Q: How does the search work technically?

Strong: "Streamlit reruns the page script when the button is clicked. It calls `fetch_data()` which reads the entire Google Sheet via the GSheets API. Pandas then filters the DataFrame to rows where 'Travel Date' equals the selected date." **Weak:** "It queries the database." → Wrong — no database query; full table download + Python filter.

Level 2: Intermediate Follow-Up

Q: Why is the full table downloaded instead of querying just the matching date?

Strong: "Google Sheets API doesn't support server-side filtering — you can only read a range or the whole sheet. There's no WHERE clause. Server-side filtering is only possible with a real database. At 700 rows, the download is ~10KB, so the performance impact is acceptable. At 100,000 rows, I'd need to migrate to PostgreSQL." **Weak:** "That's just how it works." → Shows no understanding of the limitation.

Q: What happens if no results are found?

Strong: "`filtered_data` will be an empty DataFrame. The `if not filtered_data.empty:` check catches this and shows `st.write('Sorry, No data found for the selected Travel Date.')` instead of rendering cards or charts." **Weak:** "It shows an error." → Imprecise; it's a user message, not a system error.

Level 3: Deep Technical

Q: Explain the `categorize_time()` function in detail.

Strong: "It takes a datetime string, parses it with `pd.to_datetime()`, extracts the hour (0–23), and returns a tuple: (interval_string, hour). Interval strings like '2PM - 3PM' are built from the hour value. Special cases: hour=0 → '12AM - 1AM'; hour=12 → '12PM - 1PM'. The function wraps in try/except to return ('Invalid Time', 0) for unparseable strings. The hour is returned separately so pie chart slices can be sorted chronologically." **Weak:** "It converts time to a string." → Misses the sorting mechanism and edge case handling.

Q: Why is the pie chart sorted chronologically instead of by frequency?

Strong: "The default `value_counts()` sorts by frequency (most common first). For a time distribution chart, alphabetical or frequency order is confusing — '2PM - 3PM' shouldn't appear before '1PM - 2PM' just because it has more entries. I sort by the actual hour value using `hour_mapping` to maintain chronological order, which makes the chart readable."

Level 4: Senior Engineer

Q: The `iterrows()` loop renders one card per row. What's wrong with it at scale?

Strong: "`iterrows()` iterates in pure Python, bypassing Pandas' vectorized C operations. At 700 rows it's fine (~milliseconds). At 50,000 rows, it would take seconds. The card HTML template is also regenerated for each row in Python string format — not batched. Better approach: build all card HTML in a list comprehension and join, or switch to `st.dataframe()` for large result sets. Adding pagination (show 20 per page) is the correct scaling fix."

Q: There's no input validation on the date picker. What could go wrong?

Strong: "If a user picks today's date and no entries exist, they get the 'No data found' message — fine. But there's no check for future dates (e.g., user picks 2030-01-01 — technically works, just returns nothing). More subtly: the date picker returns a Python `datetime.date` object, which is compared to `pd.to_datetime(date_input)` — this should work but could cause subtle type mismatch issues in edge cases. I should add: `if date_input > datetime.date.today(): st.warning('No future carpools registered yet.')`"

Level 5: Architecture & Scalability**Q: How would you redesign this feature for 10,000 users?**

Strong: "Three changes: (1) Fix caching — TTL-based so we don't hit Sheets on every search. (2) Add pagination — show 20 results per page with a 'Load more' button, using Streamlit's session state to track offset. (3) Migrate to FastAPI + PostgreSQL — add a `GET /api/travelers?date=2024-10-14` endpoint with a Postgres query using a `travel_date` index. This reduces the data transferred from ~100KB (full sheet) to ~2KB (matching rows only) and drops query time from ~500ms to ~5ms."

Feature 2: All Days Summary

Business Purpose: Birds-eye view of travel demand across all dates — helps identify peak travel days.

Frontend: Two pie charts (dates, destinations); auto-loads on page visit

Critical Bug: `st.cache_data.clear()` at module level — fires on every page load

Level 1: Basic**Q: What does this page show?**

Strong: "Two pie charts: distribution of travelers across all travel dates, and distribution across all destinations. It loads automatically when you visit the page — no button click required."

Level 2: Intermediate**Q: Why is `st.cache_data.clear()` at module level a problem?**

Strong: "In Streamlit, module-level code runs every time ANY user visits the page. So `st.cache_data.clear()` fires on every page load, invalidating the cache for all users. This means every

visit hits the Google Sheets API regardless of when the last call was made. If 10 users visit simultaneously, 10 API calls fire immediately. The 100 req/100s rate limit would be hit with just 100 concurrent users."

Level 3: Deep Technical

Q: How would you fix this without losing data freshness?

Strong: "Remove `st.cache_data.clear()` entirely. Add `@st.cache_data(ttl=300)` to `fetch_data()`. This caches the DataFrame for 5 minutes across all users — shared cache. When TTL expires, the next request fetches fresh data. If users need truly live data: TTL=60 (1 minute). This reduces API calls by ~90% while keeping data reasonably fresh for a carpool scheduling use case where data doesn't change second-by-second."

Feature 3: CI/CD Pipeline

Business Purpose: Automated quality assurance — ensures every code change is tested, linted, and security-scanned.

Level 1: Basic

Q: What CI/CD do you have?

Strong: "Five GitHub Actions workflows: `ci.yml` runs pytest with coverage on every push/PR. `lint.yml` runs Ruff linter and formatter check. `security-audit.yml` runs pip-audit for dependency CVEs and TruffleHog for secret scanning — triggered on push and weekly via cron. `smoke-test.yml` pings the live URL after deploy to verify HTTP 200. `update-docs.yml` creates weekly auto-PRs for documentation."

Level 2: Intermediate

Q: Why do you have 5 separate workflow files instead of one?

Strong: "Separation of concerns in CI. If tests fail, I want to know immediately without waiting for linting to finish first. Separate workflows also allow independent failure reasons — if only linting fails, the test workflow still passes and shows green. It also allows different trigger strategies: security-audit runs on a weekly cron schedule; smoke-test only runs on push to main."

Level 3: Deep Technical

Q: Why is `continue-on-error: true` a problem?

Strong: "It means the job is marked as passed even if the step fails. In `lint.yml`, if Ruff finds 50 errors, the workflow shows green. This makes the lint check purely informational — it has no quality gate effect. Any developer can push broken code and it merges fine. The fix is to remove `continue-on-error: true`, which means lint failures block the merge. You accept this tradeoff: strictness vs. the annoyance of fixing lint on every commit."

Level 4: Senior Engineer

Q: The smoke test uses `sleep 600`. What's the production-grade alternative?

Strong: "Polling loop with timeout and exponential backoff. Check every 30 seconds up to 15 minutes:

```
max_attempts=30; attempt=0
while [ $attempt -lt $max_attempts ]; do
  response=$(curl -s -o /dev/null -w "%{http_code}" https://app-url/)
  if [ $response -eq 200 ]; then echo "✅ App healthy"; exit 0; fi
  attempt=$((attempt + 1))
  sleep 30
done
echo "❌ App failed to come up"; exit 1
```

This wastes at most 15 minutes instead of always waiting 10, and exits early when the app is ready."

Level 5: Architecture

Q: How would you evolve this CI pipeline for a team of 10 engineers?

Strong: "Add branch protection rules: require all checks to pass + 1 PR review before merge to main. Add a staging environment: deploy PRs to a preview URL automatically. Add performance regression tests: compare median response time to baseline. Pin all third-party Actions to commit SHAs. Add SAST (Bandit for Python) in addition to dependency scanning. Add a code coverage threshold: fail if coverage drops below 80%."

24. Code Walkthrough Preparation

Module: **utils.py**

```
import pandas as pd

def handle_nan(value):
    """Returns 'Nil' if the value is NaN, otherwise returns the value."""
    return "Nil" if pd.isna(value) else value

def categorize_time(time_str):
    """Categorizes a time string into a 1-hour interval (e.g., '2PM - 3PM')."""
    try:
        time = pd.to_datetime(time_str)
        hour = time.hour
        if hour == 0:
            return "12AM - 1AM", 0
        elif hour < 12:
            return f"{hour}AM - {hour + 1}AM" if hour != 11 else "11AM - 12PM", hour
        elif hour == 12:
            return "12PM - 1PM", 12
        else:
            h = hour - 12
            return f"{h}PM - {h + 1}PM" if h != 11 else "11PM - 12AM", hour
    except Exception:
        return "Invalid Time", 0
```

How to explain this in an interview:

"utils.py contains two pure utility functions — no Streamlit imports, no external dependencies beyond Pandas. This design choice was intentional: by keeping utility logic separate from UI code, I can unit test it in isolation. `handle_nan` is a thin wrapper around `pd.isna()` — it returns 'Nil' for any NA value, which includes NaN, None, and `pd.NA`. `categorize_time` parses a datetime string, extracts the hour, and returns a human-readable interval string plus the raw hour integer. The hour integer is needed separately for sorting pie chart slices chronologically."

Edge cases to be ready for:

- `handle_nan(0)` → returns 0 (not "Nil") because `pd.isna(0)` is False
- `handle_nan("")` → returns "" (empty string is not NA)
- `categorize_time` at hour 11 → "11AM - 12PM" (special case in if-branch)
- `categorize_time` at hour 23 → "11PM - 12AM" (special case in else-branch)
- `categorize_time("not a time")` → "Invalid Time", 0

Likely follow-up:

"What's the bug risk in `categorize_time` for hour 23?" "The `else` branch: `h = 23 - 12 = 11`. Then `f\"{h}PM - \"{h+1}PM\"` would give `11PM - 12PM` (wrong). The correct special case `if h != 11` `else "11PM - 12AM"` handles this. This IS tested by the existing `test_categorize_time_pm` but only for hour 14 — hour 23 is an untested edge case."

Module: 1_🔍_Search_by_Date.py (Key Sections)

Section: `fetch_data()`

```
def fetch_data():
    conn = st.connection("gsheets", type=GSheetsConnection)
    return conn.read(spreadsheet=url, usecols=[2, 3, 4, 5, 6, 7])
```

How to explain:

"This creates a Streamlit connection object using the GSheets connector library, then reads columns 2 through 7 from the Google Sheet. The connection is stateless — it's recreated each call. The columns are accessed by integer index, which is fragile — if someone adds a column to the Sheet before column 2, all our data would shift. I'd fix this by reading by column name."

Section: Card rendering

```
st.markdown(f"""
    <div class="card">
        <h4>Person {count} of {total_results}</h4>
        <p><strong>👤 Name:</strong> {handle_nan(row.iloc[0])}</p>
        <p><strong>📞 Contact:</strong> {handle_nan(row.iloc[1])}</p>
        <p><strong>🕒 Travel Time:</strong> {handle_nan(row.iloc[2])}</p>
        <p><strong>📍 Destination :</strong> {handle_nan(row.iloc[3])}</p>
        <p><strong>💬 Message:</strong> {handle_nan(row.iloc[4])}</p>
    </div>
""", unsafe_allow_html=True)
```

How to explain:

"I inject custom HTML using `st.markdown` with `unsafe_allow_html=True`. The card has a gradient background, hover scale effect, and box shadow defined in the `card_style()` function above. The data comes from `row.iloc[0-4]` — `iloc` is positional indexing. I pipe each value through `handle_nan()` to replace NA values with 'Nil'. The XSS risk here is that `row.iloc[4]` is the user's 'Message' from the Google Form — if someone submits `<script>alert('xss')</script>`, it would execute. The fix is `html.escape(str(handle_nan(row.iloc[4])))` before insertion."

Module: .github/workflows/ci.yml

```
- name: Run tests with coverage
  run: |
    python -m pytest test_app.py -v --cov=. --cov-report=term-missing --cov-report=html
- name: Upload coverage report
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: coverage-report
    path: htmlcov/
    retention-days: 7
```

How to explain:

"The CI workflow runs pytest with two coverage reporters: `term-missing` prints the coverage table to the console with line numbers of uncovered code. `html` generates a visual HTML report showing exactly which lines are covered. The report is uploaded as a GitHub Actions artifact retained for 7 days — so anyone reviewing the PR can download and inspect exactly which lines aren't tested. `if: always()` means the artifact uploads even if tests fail — important for debugging failures."

25. Question Tree Expansion

Topic: Architecture

L1: What is the architecture of V_Carpool?

Streamlit frontend + Google Sheets backend + GitHub Actions CI/CD. No dedicated backend server.

L2: Why no dedicated backend server?

Google Forms → Sheets provides zero-code data ingestion. Adding a backend would add ops overhead without benefit at this scale.

L3: What are the consequences of having no backend API?

Cannot be consumed by mobile apps. Cannot add auth without middleware. Cannot implement server-side filtering. Cannot rate limit requests at the API layer.

L4: How would you add a backend without breaking the existing app?

Add FastAPI app alongside Streamlit. Streamlit calls FastAPI instead of Sheets directly. Gradual migration: first just add the `/api/travelers` endpoint, then add auth, then deprecate direct Sheets access.

L5: How would you design the API contract?

`GET /api/travelers?date=2024-10-14&destination=Mumbai&limit=20&offset=0` → `{total: 45, results: [{name: "...", time: "...", ...}]}`. Rate limiting: 10 req/min per IP. Auth: Bearer token. Response cached in Redis for 5 min per (date, destination) key.

L6: At 1M users, how does this architecture need to change?

CDN for static assets (Streamlit's JS bundle). API Gateway (Kong/NGINX) in front of FastAPI instances. Horizontal scale via Kubernetes. PostgreSQL with read replicas. Redis cluster for shared caching. Async job queue (Celery) for non-critical operations. Message queue (Kafka) for real-time updates.

Topic: Caching

L1: How does Streamlit caching work?

`@st.cache_data` caches function outputs keyed by arguments. Returns cached value if key exists and TTL hasn't expired.

L2: What's wrong with the current caching in this project?

`st.cache_data.clear()` is called on every load — destroys all cached data for all users. Equivalent to no caching.

L3: What is the correct caching strategy?

`@st.cache_data(ttl=300)` — cache for 5 minutes. Cache is shared across all users for the same function call signature. Auto-expires after TTL.

L4: What are the tradeoffs of a 5-minute TTL?

Data is up to 5 minutes stale. For a carpool app, 5 minutes is acceptable — no one needs sub-second freshness for 12-hour travel windows. If TTL too low: high API calls. If TTL too high: users see outdated results.

L5: How would you implement cache invalidation when a new form submission arrives?

Google Forms → Webhook (via Apps Script) → POST to FastAPI endpoint → FastAPI deletes the Redis cache key for the affected date. This is cache invalidation by event, not by TTL. More complex but ensures freshness on new submission.

L6: What is cache stampede and how do you prevent it?

Cache stampede: TTL expires simultaneously for many users; all send requests to origin simultaneously. Prevention: (1) Jitter in TTL — add random 0–60s to base TTL. (2) Probabilistic early recomputation (XFetch algorithm). (3) Lock on cache miss — only one request fetches, others wait.

Topic: Testing

L1: What tests do you have?

7 unit tests in `test_app.py` covering `handle_nan` and `categorize_time`.

L2: What is NOT tested?

All page logic: `fetch_data()`, card rendering, chart generation, error handling, date filtering.

L3: How would you test `fetch_data()` without hitting the real API?

Mock the connection: `unittest.mock.patch('streamlit.connection', return_value=MockConnection())` where `MockConnection().read()` returns a hardcoded DataFrame. Test that the function returns a DataFrame with the expected columns and shape.

L4: How do you test Streamlit UI components?

Two approaches: (1) `streamlit.testing.v1.AppTest` — Streamlit's official testing API that simulates user interactions without a browser. (2) Playwright/Selenium for E2E — launches a real browser, navigates to the app URL, clicks buttons, asserts DOM content.

L5: What is a contract test and does this project need one?

Contract test verifies that the Google Sheets API returns data in the expected schema. This project should have one: assert that the DataFrame returned by `conn.read()` has at least 6 columns, that column 5 is parseable as datetime, etc. This would catch column renames in the Sheet before they cause silent failures.

L6: How would you implement mutation testing?

Use `mutmut` or `cosmic-ray`. They introduce small code mutations (e.g., change `==` to `!=` in `handle_nan`) and verify that at least one test fails. If tests pass despite the mutation, they're not testing the right behavior. This measures test effectiveness beyond coverage %.

26. Project Structure & Code Organization

Folder Walkthrough

```

Carpool_App/
├─ main.py           ← App entry point. Home page, navigation buttons.
                     No business logic – just routing and welcome text.
├─ utils.py          ← Pure utility module. No Streamlit, no external API.
                     Designed for testability. Contains handle_nan, categorize_time.
├─ test_app.py       ← Unit tests. Imports only from utils.py.
                     7 tests; pytest + pytest-cov.
├─ requirements.txt  ← Python dependencies. Also pins dev tools (ruff, pytest, pip-audit).
├─ packages.txt      ← OS-level packages (currently empty). Required by Streamlit Cloud.
├─ pages/            ← Streamlit multi-page routing. Files here become sidebar menu items.
                     Naming: {order}_{emoji}_{Name}.py
    ├─ 1_🔍_Search_by_Date.py ← Core feature. Date picker, GSheets fetch, card display.
    ├─ 2_📊_All_Days_Summary.py ← Overview dashboard. Aggregate pie charts across all data.
    └─ 3_👤_Credits_Page.py   ← Developer profile. Static content.
├─ .devcontainer/
    └─ devcontainer.json ← GitHub Codespaces config. Python 3.11 image, auto-installs requirements.
                           auto-starts Streamlit on port 8501.
└─ .github/workflows/ ← CI/CD automation.
    ├─ ci.yml          ← pytest + coverage. Triggers on push/PR to main/develop.
    ├─ lint.yml        ← Ruff linter + formatter check. continue-on-error (decorative).
    ├─ security-audit.yml ← pip-audit + TruffleHog. Weekly cron + on push.
    ├─ smoke-test.yml  ← HTTP health check post-deploy. On push to main only.
    └─ update-docs.yml ← Auto-generates structure.txt; creates PR if changed. Weekly
  
```

Separation of Concerns Analysis

Concern	Location	Correct?
UI rendering	<code>pages/</code> files	✓
Business logic	Mixed in <code>pages/</code> files	✗ Should be in <code>utils.py</code>
Data fetching	<code>fetch_data()</code> in each page file	✗ Duplicated; should be in <code>utils.py</code>
Configuration	Hardcoded in each page file	✗ Should be <code>config.py</code>
Testing	<code>test_app.py</code>	✓ (but thin)
Utility functions	<code>utils.py</code>	✓

Interview Explanation of Structure

"The project follows Streamlit's prescribed multi-page structure: `main.py` is the entry point, and the `pages/` directory contains additional pages automatically registered by Streamlit. I separated pure utility functions into `utils.py` so they could be tested independently without running the Streamlit server. The CI workflows are in `.github/workflows/` with one file per concern — tests, linting, security, deployment verification. The `devcontainer.json` allows anyone to open the project in GitHub Codespaces and start developing immediately."

27. API Deep Dive

FACT: This project exposes no internal REST APIs. It consumes one external API: the Google Sheets API via the `st-gsheets-connection` connector.

External API: Google Sheets via `st-gsheets-connection`

Purpose: Read travel registration data from the Google Sheet

Request (internal):

```
conn = st.connection("gsheets", type=GSheetsConnection)
df = conn.read(
    spreadsheet="https://docs.google.com/spreadsheets/d/1fbEmtrmVu9heYMfL5P1a9qJ5q0xT3tFj
    usecols=[2, 3, 4, 5, 6, 7] # columns to read
)
```

Under the hood:

- The connector authenticates via service account credentials from Streamlit Secrets
- Makes HTTP GET to Google Sheets API v4: `GET /v4/spreadsheets/{id}/values/{range}`
- Returns JSON array of rows, converted to Pandas DataFrame

Response (DataFrame):

	Name	Phone	Travel Time	Travel Date	Destination	Notes
0	Alice	9876543210	2024-10-14 14:00:00	2024-10-14	Mumbai	Window seat pr
1	Bob	9123456789	2024-10-14 09:00:00	2024-10-14	Pune	None
...						

Authentication: Service account OAuth2 (credentials in Streamlit Secrets — not in repo)

Rate Limits: 100 reads per 100 seconds per project; 500 reads per 100 seconds per user

Error Handling: Wrapped in `try/except Exception` — catches all errors including 429 (rate limit), 403 (auth failure), 500 (Sheets outage)

Interview Questions on API Design:

Q: If you were to build your own API, what would it look like?

```
GET /api/travelers?date=2024-10-14
Response: {
  "date": "2024-10-14",
  "count": 45,
  "travelers": [
    {"name": "Alice", "time": "14:00", "destination": "Mumbai", "notes": "Window pr
    ...
  ],
  "time_distribution": {"2PM-3PM": 15, "3PM-4PM": 12, ...},
  "destination_distribution": {"Mumbai": 20, "Pune": 15, ...}
}
```

Rate limiting: 10 req/min per IP. Auth: Bearer token from Google OAuth. Cache-Control: max-age=300.

Q: What HTTP status codes should this API return?

200 OK (results found), 200 with empty list (no results for date), 400 Bad Request (invalid date format), 401 Unauthorized (missing/invalid token), 429 Too Many Requests (rate limit), 500 Internal Server Error (DB or GSheets failure), 503 Service Unavailable (circuit breaker open).

Q: How would you version the API?

URL versioning: `/api/v1/travelers`, `/api/v2/travelers`. V2 could add pagination, richer response format. Maintain v1 for backward compatibility with any existing consumers (mobile app, bots).

28. Database Query Cross-Examination

FACT: There is no SQL/NoSQL database. Data operations are Pandas DataFrame operations on in-memory data fetched from Google Sheets.

Query Equivalent 1: Filter by Date

Operation: `data[data['Travel Date'] == pd.to_datetime(date_input)]`

SQL Equivalent:

```
SELECT name, phone, travel_time, destination, notes
FROM travelers
WHERE travel_date = '2024-10-14';
```

Performance:

- **Current (Pandas):** O(n) full table scan; 700 rows → negligible
- **At 100k rows:** ~10ms in Pandas; ~1ms with PostgreSQL index scan
- **PostgreSQL index:** `CREATE INDEX idx_travel_date ON travelers(travel_date);`

Interview Q: What index would you use?

“B-tree index on `travel_date` — standard for equality and range queries. If queries often filter by date AND destination together, a composite index `(travel_date, destination)` would be more efficient — PostgreSQL can use it for date-only queries too (leftmost prefix rule).”

Query Equivalent 2: Value Counts by Date

Operation: `data['Travel Date'].value_counts()`

SQL Equivalent:

```
SELECT travel_date, COUNT(*) as traveler_count
FROM travelers
GROUP BY travel_date
ORDER BY traveler_count DESC;
```

Performance: Full table scan required for GROUP BY. Index on `travel_date` helps if combined with query planner's index-only scan.

Query Equivalent 3: Time Distribution

Operation: `filtered_data[col].apply(categorize_time)` → `value_counts()`

SQL Equivalent:

```
SELECT
CASE
WHEN EXTRACT(hour FROM travel_time) = 0 THEN '12AM - 1AM'
WHEN EXTRACT(hour FROM travel_time) < 12 THEN
    CONCAT(EXTRACT(hour FROM travel_time)::text, 'AM - ',
            (EXTRACT(hour FROM travel_time) + 1)::text, 'AM')
-- ... etc
END as time_slot,
COUNT(*) as count
FROM travelers
WHERE travel_date = '2024-10-14'
GROUP BY time_slot
ORDER BY MIN(EXTRACT(hour FROM travel_time));
```

Interview Q: Why do this in Pandas instead of SQL?

"Because there's no SQL database — data comes from Sheets. In a PostgreSQL migration, pushing this aggregation to the database would be much faster, especially at scale. Database aggregations benefit from indexes and avoid transferring raw data to the application layer."

Interview Questions on DB Design

Q: How would you design the PostgreSQL schema?

```
CREATE TABLE travelers (
  id          BIGSERIAL PRIMARY KEY,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  name        VARCHAR(255) NOT NULL,
  phone       VARCHAR(20), -- nullable; PII
  travel_time TIMETZ,
  travel_date DATE NOT NULL,
  destination VARCHAR(255),
  notes       TEXT,
  form_response_id VARCHAR(255) UNIQUE -- dedup by form response
);

CREATE INDEX idx_travelers_date ON travelers(travel_date);
CREATE INDEX idx_travelers_date_dest ON travelers(travel_date, destination);
```

Q: What constraints would you add?

NOT NULL on `name` and `travel_date` (required for core functionality). UNIQUE on `form_response_id` to prevent duplicate submissions. CHECK constraint on `phone` format (regex). Partial index: `WHERE phone IS NOT NULL` for phone-based matching queries.

Q: How would you handle data migrations?

"Alembic for schema migrations with FastAPI/SQLAlchemy. Each migration is a versioned Python file. Run migrations as part of the deployment pipeline, before the app starts (`alembic upgrade head`). Never modify production data in a migration — always additive changes (add column, add index) before removing old columns."

Q: What are the transaction considerations?

"Write operations (new traveler registration) should be a single INSERT — atomic by nature. Batch imports from Sheets during migration should be wrapped in a transaction: `BEGIN; INSERT ...; COMMIT;` so a partial import doesn't leave data in an inconsistent state."

Hidden Interview Traps

Trap 1: "You said 300+ daily requests — how did you measure that?"

Why dangerous: If you can't explain the measurement, the stat is meaningless.

Weak answer: "That's in the README, the app was really popular."

Strong answer: "Streamlit Community Cloud provides a viewer count dashboard. At peak (Diwali 2024), the dashboard showed 300+ daily active viewers. A 'request' in Streamlit is a page load or interactive event. I'd clarify on my resume: 300+ daily active viewers, not HTTP requests in the traditional API sense."

Trap 2: "Walk me through your CI pipeline and show me that it actually works"

Why dangerous: `continue-on-error: true` means failures are invisible. If the interviewer checks the Actions tab on GitHub, they'll see green checks even when Ruff finds errors.

Weak answer: "It runs tests, linting, and security scanning on every push."

Strong answer: "The test pipeline (ci.yml) is a real quality gate — it fails if tests fail. The lint and security pipelines currently use `continue-on-error: true`, which means they log failures but don't block merges — that's a known weakness I'd fix by removing that flag. The smoke test genuinely blocks deployment if the live URL returns non-200."

Trap 3: "Is this project secure enough to handle real user data?"

Why dangerous: The answer is no, and you should say so proactively.

Weak answer: "Yes, I have TruffleHog and pip-audit."

Strong answer: "Honestly, not yet. I've identified three issues: (1) The Google Sheet is publicly accessible, exposing 1,100 users' names and phone numbers — I'd restrict it to a service account only. (2) User-submitted 'Notes' are rendered in HTML without sanitization — XSS risk fixed with `html.escape()`. (3) There's no authentication — anyone can view anyone's data. For a production launch, I'd add at least OAuth2 with university email verification. I consider this v1 — it solved the coordination problem, but v2 needs the security hardening."

Trap 4: "Your code has hundreds of lines of comments — why?"

Why dangerous: The code files are 50% commented-out old code. This looks unprofessional.

Weak answer: "I kept it for reference."

Strong answer: "That's dead code from earlier iterations — I was iterating on the feature and left the old versions in-place instead of relying on git history. That's a bad practice I've since corrected in my workflow: git history is the right place for dead code; the active file should only contain live code. I'd delete all commented-out sections in a quick cleanup pass."

Trap 5: "Why did you access columns by index [2,3,4,5,6,7] instead of by name?"

Why dangerous: Shows fragility and lack of defensive programming.

Weak answer: "That's how the tutorial showed it."

Strong answer: "It was quick and worked at the time. The risk is that adding a column to the Google Sheet before column 2 would silently shift all indices and display wrong data. The correct fix is `usecols=['Name', 'Phone', 'Travel Time', 'Travel Date', 'Destination', 'Notes']` and accessing by column name. I'd move these to a `config.py` constant so there's one place to update if the schema changes."

Trap 6: "Your README says '90% automated logistics' — prove it"

Why dangerous: This is a vague marketing claim with no technical basis.

Weak answer: "The app automates finding partners."

Strong answer: "That bullet is vague and I should remove it. What I can defend: the data ingestion is 100% automated (Forms → Sheets, zero manual work). The matchmaking discovery is self-service. I'd replace this with: 'Designed a zero-ops data ingestion pipeline using Google Forms + Sheets API, eliminating all manual data entry' — which is specific and defensible."

Trap 7: "Can your app handle the next semester's carpool window?"

Why dangerous: This tests scaling awareness.

Weak answer: "Yes, it handled Diwali so it can handle more."

Strong answer: "It handled Diwali with ~300 daily users. For the next semester, if usage is similar, yes. But I've identified a caching bug — `st.cache_data.clear()` on every load — that would hit the Google Sheets API rate limit (100 req/100s) if we see 100+ concurrent users. My immediate fix is TTL-based caching. If we anticipate 10x growth, I'd migrate to PostgreSQL before the next major break."

Trap 8: "The TruffleHog action is `@main` — explain why that's a security issue"

Why dangerous: Tests actual security engineering knowledge.

Weak answer: "I should probably update it."

Strong answer: "GitHub Actions pinned to a tag like `@main` or `@v1` are vulnerable to supply chain attacks. If the TruffleHog repository's main branch is compromised — even temporarily — a malicious update could run arbitrary code in my CI pipeline with full access to the repository secrets. The fix is pinning to a commit SHA: `trufflesecurity/trufflehog@a1b2c3d4e5...` — a SHA is content-addressed and won't change even if the upstream repo is compromised. I'd use Dependabot to keep the SHA updated safely."

Final Assessment

"If I were interviewing this candidate, would this project increase confidence in hiring them?"

Hire Confidence Score: 6.5/10

Biggest Strengths:

1. Real production system with real users — most student projects are toys
2. 5 CI/CD workflows including security scanning — shows engineering maturity beyond the code
3. Proactive identification of own bugs and security issues — self-awareness is rare
4. Extracted testable utility functions — shows design instinct
5. Can explain the project at multiple levels of depth with honesty about tradeoffs

Biggest Concerns:

1. Dead code everywhere — first thing a code reviewer notices; suggests low code hygiene
2. Caching bug — a fundamental misunderstanding that went unnoticed
3. PII exposure — not acceptable in production; shows security was an afterthought
4. 0% test coverage of page logic — only trivial utilities tested
5. Resume contains non-defensible claims (30% faster commute, 90% automated) — could undermine credibility

What would make this project stand out among hundreds of student projects:

1. Fix the caching bug + delete dead code + PII masking (3 hours of work, massive credibility increase)
2. Add AI matchmaking — Gemini API integration for natural language search
3. Migrate to FastAPI + PostgreSQL and document the migration (demonstrates real backend engineering)
4. Add Sentry + structured logging (shows production thinking)
5. Rewrite resume bullets to be specific and defensible

Bottom line: This candidate built something real, deployed it, got users, and thought about CI/CD beyond most peers. They're also honest about limitations when pressed. With the identified cleanup items addressed, this becomes a genuinely strong portfolio project for SDE/Full-Stack/Data roles.

Executive Summary

Top Strengths

Strength	Evidence
Real production system	1,100+ users, 700+ form entries, live URL
CI/CD maturity	5 GitHub Actions workflows including security scanning
Engineering self-awareness	Can identify and explain own bugs and limitations
Utility function design	<code>utils.py</code> properly separated and testable
Real problem solved	Measurable adoption proves product-market fit

Top Weaknesses

Weakness	Severity	Fix Time
PII publicly exposed	● Critical	2 hours
Caching bug defeats performance	● High	30 minutes
50% dead code in all page files	● High	30 minutes
0% test coverage of core pages	● High	1 day
CI checks not blocking merges	● Medium	5 minutes

Top Interview Talking Points

1. "I built this for real users — 1,100 students adopted it"
2. "I have 5 CI/CD pipelines including TruffleHog secret scanning"
3. "I identified a caching bug where `cache_data.clear()` was called on every load"
4. "I identified a PII exposure risk — the Google Sheet was publicly accessible"
5. "The utility functions are in a separate `utils.py` so they can be unit tested independently"
6. "I used the Google Forms → Sheets pipeline as zero-code data ingestion"
7. "My roadmap includes AI matchmaking and a FastAPI + PostgreSQL migration"

Top Production Improvements

Priority	Improvement	Time
1	Fix caching bug → <code>@st.cache_data(ttl=300)</code>	30 min
2	Mask PII in UI; restrict Sheet to service account	2 hr
3	Delete dead code	30 min
4	Remove <code>continue-on-error: true</code> from CI	5 min
5	Add <code>html.escape()</code> for XSS prevention	1 hr
6	Add Sentry error tracking	2 hr
7	Add structured logging	2 hr
8	Mock GSheets in tests; add page coverage	1 day

Final Readiness Assessment

Role	Readiness	Notes
Entry-Level SDE	7/10	Strong for entry; clean up code first
Data Analyst	7/10	Add Plotly charts; strong data story
Full-Stack Engineer	5/10	Needs FastAPI + proper backend story
Backend Engineer	4/10	No REST API experience demonstrated yet
AI/ML Engineer	3/10	Implement the matchmaking feature first
DevOps/SRE	6/10	5 workflows is strong; needs Docker
Security Engineer	5/10	Good awareness; execution needs work

One-line verdict: This is a legitimate v1 production system that needs 3 days of disciplined cleanup and 1 new feature to become a genuinely competitive portfolio project for SDE/full-stack/data roles.